Esercizio 1)

We have stressed the need for an operating system to make efficient use of the computing hardware. When is it appropriate for the operating system to forsake this principle and to "waste" resources? Why is such a system not really wasteful?

Answer:

Single-user systems should maximize use of the system for the user. A GUI might "waste" CPU cycles, but it optimizes the user's interaction with the system.

What is the main difficulty that a programmer must overcome in writing an operating system for a real-time environment?

Answer:

The main difficulty is keeping the operating system within the fixed time constraints of a real-time system. If the system does not complete a task in a certain time frame, it may cause a breakdown of the entire system it is running. Therefore when writing an operating system for a real-time system, the writer must be sure that his scheduling schemes don't allow response time to exceed the time constraint.

Keeping in mind the various definitions of **operating system,** consider whether the operating system should include applications such as Web browsers and mail programs. Argue both that it should and that it should not, and support your answers.

Answer:

An argument in favor of including popular applications with the operating system is that if the application is embedded within the operating system, it is likely to be better able to take advantage of features in the kernel and therefore have performance advantages over an application that runs outside of the kernel. Arguments against embedding applications within the operating system typically dominate however: (1) the applications are applications - and not part of an operating system, (2) any performance benefits of running within the kernel are offset by security vulnerabilities, (3) it leads to a bloated operating system.

How does the distinction between kernel mode and user mode function as a rudimentary form of protection (security) system?

Answer:

The distinction between kernel mode and user mode provides a rudimentary form of protection in the following manner. Certain instructions could be executed only when the CPU is in kernel mode. Similarly, hardware devices could be accessed only when the program is executing in kernel mode. Control over when interrupts could be enabled or disabled is also possible only when the CPU is in kernel mode. Consequently, the CPU has very limited capability when executing in user mode, thereby enforcing protection of critical resources.

Esercizio 2)

A CPU-scheduling algorithm determines an order for the execution of its scheduled processes. Given n processes to be scheduled on one processor, how many different schedules are possible? Give a formula in terms of n.

Answer:

$n!$ (n factorial = $n \times n-1 \times n-2 \times \ldots \times 2 \times 1$).

Esercizio 3)

```
for (int i = 0; i < n; i++) {
   // first find a thread that can finish
   for (int j = 0; j < n; j++) {
      if (!finish[j]) {
         boolean temp = true;
         for (int k = 0; k < m; k++) {
            if (need[j][k] > work[k])
               temp = false;
         }

         if (temp) { // if this thread can finish
            finish[j] = true;
            for (int x = 0; x < m; x++)
               work[x] += work[j][x];
         }
      }
   }
}
```

Figure 7.1 Banker's algorithm safety algorithm.

- N è il numero di processi;
- M il numero di risorse;
- work[k] è il numero di risorse k disponibili nel sistema
- need[j][k] numero di risorse k che il processo j necessita
- work[j][k] numero di risorse k che il processo j utilizza per «lavorare»

As can be seen, the nested outer loops—both of which loop through n times—provide the n^2 performance. Within these outer loops are two sequential inner loops which loop m times. The big-oh of this algorithm is therefore $O(m \times n^2)$.

Esercizio 4)

Si consideri un sistema contenente tre risorse A, B e C, rispettivamente con 10, 3 e 8 istanze. Si assuma che a un dato istante t siano in esecuzione i processi P_0, P_1, P_2 e P_3, e che le allocazioni correnti delle risorse ai processi e il numero massimo di richieste di allocazione siano descritte dalle seguenti matrici.

Assegnate

	A	B	C
P_0	3	1	2
P_1	2	0	1
P_2	0	2	1
P_3	1	0	1

Massimo

	A	B	C
P_0	7	2	4
P_1	2	1	3
P_2	5	2	5
P_3	3	0	2

Applicando l'algoritmo del banchiere si risponda alle seguenti domande:

1. La sequenza di processi $< P_0, P_1, P_2, P_3 >$ è sicura?
2. La sequenza di processi $< P_0, P_2, P_1, P_3 >$ è sicura?
3. La sequenza di processi $< P_3, P_2, P_0, P_1 >$ è sicura?
4. La sequenza di processi $< P_3, P_1, P_2, P_0 >$ è sicura?
5. Nell'istante t il sistema si trova in uno stato sicuro?

Se la risposta all'ultima domanda è affermativa, indicare quali delle seguenti richieste di risorse (ognuna da considerare partendo dallo stato all'istante t, indicando con R_i una richiesta fatta dal processo P_i) possono essere soddisfatte, sempre applicando l'algoritmo del banchiere:

6. $R_1 = (0, 0, 1)$,
7. $R_1 = (2, 0, 0)$,
8. $R_1 = (0, 0, 2)$.

Svolgimento: Per applicare l'algoritmo del banchiere è necessario calcolare la matrice Necessità che descrive il massimo numero residuo di istanze allocabili ai processi e il vettore Disponibili che contiene il numero di istanze attualmente disponibili nel modo seguente.

- Necessità si determina calcolando la differenza membro a membro tra le matrici Massimo e Assegnate, ottenendo il risultato seguente.

Necessità

	A	B	C
P_0	4	1	2
P_1	0	1	2
P_2	5	0	4
P_3	2	0	1

- Disponibili si calcola invece sottraendo, risorsa per risorsa, la somma degli elementi in colonna di Assegnate dal vettore che ha per componenti i numeri totali delle istanze per le varie risorse, ottenendo $(4, 0, 3)$.

Una volta determinate tutte le strutture dati, è possibile considerare le prime cinque domande dell'esercizio.

1. La sequenza $< P_0, P_1, P_2, P_3 >$ non è sicura in quanto il primo dei suoi processi (P_0) richiede un'istanza addizionale della risorsa B per portare a termine la propria esecuzione, ma di tale risorsa non sono disponibili istanze libere.

2. La sequenza $< P_0, P_2, P_1, P_3 >$ non è sicura in quanto il primo dei suoi processi è P_0, e dunque valgono le stesse considerazioni del punto precedente.

3. La sequenza $< P_3, P_2, P_0, P_1 >$ è sicura in quanto risulta possibile considerare, nell'ordine, tutti i suoi processi garantendo che ognuno trovi disponibili abbastanza istanze per essere eseguito. Più precisamente, P_3 individua la riga $[2, 0, 1]$ della matrice Necessità, e i suoi valori sono–componente per componente–minori o uguali di quelli del vettore Disponibili. Il processo P_3 risulta quindi eseguibile per primo, e alla sua terminazione la quantità di istanze disponibili sarà determinata aggiungendo a Disponibili la riga della matrice Assegnate individuata da P_3, ottenendo $[5, 0, 4]$. Lo stesso procedimento si può applicare, nell'ordine, ai processi restanti nella sequenza, ottenendo via via le istanze disponibili illustrate qui sotto.

	Disponibili		
	A	B	C
	4	0	3
P_3	5	0	4
P_2	5	2	5
P_0	8	3	7
P_1	10	3	8

4. La sequenza $< P_3, P_1, P_2, P_0 >$ non è sicura: come visto al punto precedente, dopo aver eseguito P_3 non riuslteranno libere istanze della risorsa R_1, e dunque non sarà possibile eseguire il processo P_1.

5. All'istante t il sistema si deve trovare in uno stato sicuro, in quanto esiste almeno una sequenza sicura (quella determinata al precedente punto 3.

Essendo la risposta alla domanda 5 affermativa, si possono considerare le rimanenti domande, valutando la soddisfacibilità delle richieste corrispondenti.

6. La richiesta $R_1 = (0, 0, 1)$ di un'istanza di R_3 da parte del processo P_1 è compatibile con la disponibilità di risorse, in quanto sono libere 3 istanze dell'istanza richiesta. Inoltre la richiesta è compatibile con i valori indicati nella seconda riga della matrice Necessità. In casi come questo, l'algoritmo del banchiere prevede di simulare l'allocazione delle risorse richieste verificando che lo stato risultante risulti sicuro. Tale stato si ottiene aggiungendo il vettore R_1 alla riga corrispondente a P_1 rispettivamente nella matrice Assegnate e sottraendolo sia dal vettore Disponibili, sia dalla riga corrispondente a P_1 della matrice Necessità. Pertanto le matrici che descriverebbero lo stato del sistema una volta allocata l'istanza richiesta sono le seguenti.

	Assegnate		
	A	B	C
P_0	3	1	2
P_1	2	0	2
P_2	0	2	1
P_3	1	0	1

	Massimo		
	A	B	C
P_0	7	2	4
P_1	2	1	3
P_2	5	2	5
P_3	3	0	2

	Necessità		
	A	B	C
P_0	4	1	2
P_1	0	1	1
P_2	5	0	4
P_3	2	0	1

	Disponibili		
	A	B	C
	4	0	2

Si verifica facilmente che l'unico processo che potrebbe essere portato a termine con le nuove disponibilità di risorse sarebbe P_3. Al termine di quest'ultimo il vettore Disponibili diventerebbe $(5,0,3)$ e nessun altro processo potrebbe essere considerato, in quanto P_0 e P_1 necessitano di un'istanza di B e P_2 necessita di 4 istanze di C. Pertanto, non esistendo una sequenza sicura, il soddisfacimento della richiesta porterebbe il sistema in uno stato non sicuro. Per questo motivo la richiesta non può essere soddisfatta.

7. La richiesta $R_1 = (2,0,0)$ di due istanze di R_1 da parte del processo P_1 potrebbe essere soddisfatta in quanto sono disponibili quattro istanze della risorsa richiesta, ma non può essere presa in considerazione in quanto la matrice Necessità indica che P_1 non può richiedere istanze addizionali di tale risorsa.

8. La richiesta $R_1 = (0,0,2)$ è sia compatibile con le risorse a disposizione, sia con il numero massimo di richieste aggiuntive indicate in Necessità. Per verificare l'attuabilità della richiesta bisogna dunque procedere come per il caso (f), considerando lo stato che si otterrebbe assegnando due istanze di C a P_1. Si nota però che è sufficiente notare come R_1 voglia accedere esattamente a un'istanza in più della richiesta considerata nella domanda (f), che già era stata dimostrata non soddisfacibile.

Pertanto nessuna delle tre richieste aggiuntive di risorse potrà essere soddisfatta.

Esercizio 5)

Si consideri un sistema contenente quattro risorse A, B, C e D, rispettivamente con 8, 2, 7 e 2 istanze, allocate a un dato istante t ai processi $P_0,\ldots,P_4$ nel modo descritto dalle seguenti matrici.

	Assegnate					Massimo			
	A	B	C	D		A	B	C	D
P_0	1	0	1	1	P_0	1	1	1	2
P_1	2	0	0	0	P_1	3	1	0	1
P_2	1	1	1	0	P_2	1	1	3	1
P_3	0	1	2	0	P_3	1	2	2	0
P_4	3	0	1	1	P_4	4	0	3	1

Nelle soluzioni si considera come valore Massimo di P4 in C non 3 ma 2.

Indicando con $R_i = (a,b,c,d)$ una richiesta rispettivamente di a istanze della risorsa A, b istanze della risorsa B e via discorrendo, fatta dal processo P_i, si considerino le seguenti richieste:

1. $R_0 = (1,0,1,0)$,
2. $R_1 = (1,0,0,0)$,
3. $R_2 = (0,0,1,0)$,
4. $R_3 = (1,1,0,0)$,
5. $R_2 = (0,0,2,0)$.

Applicando l'algoritmo del banchiere, determinare se le richieste siano o meno soddisfacibili, giustificando la risposta. Ciascuna richiesta deve essere considerata in modo indipendente dalle altre. Alla luce delle risposte date al punto precedente, il sistema al tempo t si trovava in uno stato sicuro?

Svolgimento: Per poter applicare l'algoritmo del banchiere è necessario calcolare la matrice Necessità e il vettore Disponibili, che indicano rispettivamente il numero massimo di istanze delle varie risorse che i vari processi necessitano per poter terminare la propria esecuzione e il numero di istanze disponibili per ogni risorsa. La matrice Necessità si ottiene sottraendo Allocate da Massimo, ottenendo la seguente matrice.

Necessità

	A	B	C	D
P_0	0	1	0	1
P_1	1	1	0	1
P_2	0	0	2	1
P_3	1	1	0	0
P_4	1	0	1	0

Il vettore Disponibli si ottiene invece considerando una risorsa alla volta e sottraendo dal numero di istanze totali il numero di istanza allocate ai vari processi, ottenendo Disponibili $= (1,0,2,0)$. Prima di poter applicare l'algoritmo del banchiere al fine di valutare la validità delle varie richieste di risorse, è necessario verificare che il sistema si trovi in uno stato sicuro. Dalla tabella successiva si vede come la sequenza $< P_4, P_2, P_1, P_0, P_3 >$ sia sicura, e quindi anche il sistema si trovi all'istante t in uno stato sicuro.

Disponibili

	A	B	C	D
	1	0	2	0
P_4	4	0	3	1
P_2	5	1	4	1
P_1	7	1	4	1
P_0	8	1	5	2
P_3	8	2	7	2

È dunque possibile valutare le richieste di ulteriori allocazioni.

1. La richiesta $R_0 = (1,0,1,0)$ è compatibile con la quantità di istanze disponibili ($(1,0,1,0) \leq (1,0,2,0) =$ Disponibili), ma non con la riga corrispondente a P_0 della matrice Necessità ($(1,0,1,0) \not\leq (0,1,0,1) =$ Necessità$_0$)[3]. Pertanto la richiesta deve essere considerata invalida.

2. La richiesta $R_1 = (1,0,0,0)$ è sia compatibile con le risorse disponibili, sia con le necessità del processo ($(1,0,0,0) \leq (1,0,2,0) =$ Disponibili e $(1,0,0,0) \leq (1,1,0,1) =$ Necessità$_1$). Per valutare se sia possibile allocare le istanze addizionali richieste è dunque necessario verificare se tale azione manterrebbe il sistema in uno stato sicuro. Il nuovo stato sarebbe descritto dal vettore Disponibili$=(0,0,2,0)$ e dalle seguenti matrici.

Assegnate

	A	B	C	D
P_0	1	0	1	1
P_1	3	0	0	0
P_2	1	1	1	0
P_3	0	1	2	0
P_4	3	0	1	1

Necessità

	A	B	C	D
P_0	0	1	0	1
P_1	0	1	0	1
P_2	0	0	2	1
P_3	1	1	0	0
P_4	1	0	1	0

Si verifica facilmente che nessuna riga della nuova matrice Necessità è minore o uguale a Disponibili, a significare che in questo nuovo stato non vi è alcun processo in grado di terminare. Pertanto la richiesta non può essere soddisfatta in quanto porterebbe il sistema in uno stato non sicuro.

3. La richiesta $R_2 = (0,0,1,0)$ è sia compatibile con le risorse disponibili, sia con le necessità del processo ($(0,0,1,0) \leq (1,0,2,0) =$ Disponibili e $(0,0,1,0) \leq (0,0,2,1) =$ Necessità$_2$). Anche in questo caso è quindi necessario verificare la sicurezza dello stato in cui si troverebbe il sistema se le risorse venissero assegnate. In tal caso si avrebbe Disponibili $= (1,0,1,0)$, e le matrici che descrivono allocazioni e necessità muterebbero come segue.

Assegnate

	A	B	C	D
P_0	1	0	1	1
P_1	2	0	0	0
P_2	1	1	2	0
P_3	0	1	2	0
P_4	3	0	1	1

Necessità

	A	B	C	D
P_0	0	1	0	1
P_1	1	1	0	1
P_2	0	0	1	1
P_3	1	1	0	0
P_4	1	0	1	0

[3] Negli esercizi la notazione M_i, dove M indica una matrice, denota il vettore ottenuto considerando l'i-esima riga di M.

Come si può verificare dalla tabella seguente, in questo nuovo stato la sequenza $< P_4, P_2, P_0, P_1, P_3 >$ è sicura.

	Disponibili			
	A	B	C	D
	1	0	1	0
P_4	4	0	2	1
P_2	5	1	4	1
P_0	6	1	5	2
P_1	8	1	5	2
P_3	8	2	7	2

La richiesta può quindi essere soddisfatta in quanto lascia il sistema in uno stato sicuro.

4. La richiesta $R_3 = (1,1,0,0)$ deve essere lasciata in sospeso: siccome

$$(1,1,0,0) \not\leq (1,0,2,0) = \text{Disponibili},$$

non vi sono abbastanza risorse disponibili.

5. La richiesta $R_2 = (0,0,2,0)$ è sia compatibile con le risorse disponibili, sia con le necessità del processo $((0,0,2,0) \leq (1,0,2,0) = \text{Disponibili}$ e $(0,0,2,0) \leq (0,0,2,1) = \text{Necessità}_2)$. Soddisfacendo questa richiesta si avrebbe Disponibili $= (1,0,0,0)$ e il sistema sarebbe descritto dalle seguenti matrici.

	Assegnate					Necessità			
	A	B	C	D		A	B	C	D
P_0	1	0	1	1	P_0	0	1	0	1
P_1	2	0	0	0	P_1	1	1	0	1
P_2	1	1	3	0	P_2	0	0	0	1
P_3	0	1	2	0	P_3	1	1	0	0
P_4	3	0	1	1	P_4	1	0	1	0

Come nel caso 2, si verifica che in questo nuovo stato non sarebbe possibile portare a termine l'esecuzione di alcun processo. La richiesta deve quindi essere lasciata in sospeso perché il suo soddisfacimento porterebbe il sistema in uno stato non sicuro.

Infine, l'ultima domanda è malposta: non ha infatti senso pensare di poter valutare la sicurezza del sistema all'istante t in base al fatto che risultino o meno soddisfacibili le richieste (1–5). Nella fattispecie, il sistema si trova in uno stato sicuro, ma ciò è stato verificato sulla base dei contenuti del vettore Disponibili e delle matrici Assegnate e Necessità all'istante t.

Esercizio 6)

- P1 risveglia P3 che incrementa x e poi risveglia P1 che stampa 101
- Sì, perché P2 e P3 accedono in contemporanea a x (la cui modifica è regolata da due semafori distinti, entrambi "aperti")

non sapere se facciamo x-1 o x+1 intende che non sappiamo chi sia il primo ad eseguire il write(x) per aggiornare il valore di x.

Esercizio 7)

Inizialmente uno qualsiasi tra P1 e P2 può essere selezionato per l'esecuzione della prima istruzione.
Il processo P1 deve attendere l'esecuzione di signal(s2) da parte di P2 e viceversa P2 attende l'esecuzione di signal(s1). L'esecuzione delle prime due stampe "P" in P1 e "A" P2 può avvenire in qualsiasi ordine. Dopo aver stampato "P", P1 attende che P2 esegua signal(s2) e quindi stampa "R" e "segnala" sul semaforo s1 sul quale attende P2. Solo dopo la signal(s1) eseguita da P1, P2 può stampare "I". I possibili output sono quindi due: la stringa "PARI" e la stringa "APRI".

Esercizio 8)

1. Stampa la sequenza infinita dei numeri naturali a partire da 1: 1 2 3 4 ....
2. Stampa la sequenza infinita dei numeri naturali a partire da 0: 0 1 2 3 4 ....
3. P3 azzera **x** in modo non-deterministico durante l'esecuzione di P1 e P2. Vengono stampate sequenze del tipo:

(0) 1 2 ..... n1 (0) 1 2 ... n2 (0) 1 2 .....

dove ni >=0 (se ni=0 la sottosequenza corrispondente è vuota).
Ad es.:

0 0 0 ....
1 1 1 ...
0 1 2 0 1 2 3 .....

M garantisce mutua esclusione per l'accesso ad **x**.

nel caso (3) dopo che P3 eseguirà signal(M), riavviando P1, poi sarà sospeso in attesa di eseguire wait(M) per tempo non noto, non sappiamo quando tornerà running per eseguire wait(M), dunque P1 e P2 continueranno a scrivere la sequenza di numeri. Quando P3 eseguirà wait(M) poi bloccherà il ciclo P1-P2, perchè un loro wait(M) non potrà essere soddisfatto, dunque P3 tornerà running (P1 e P2 sospesi) e azzererà x. Se il signal(M) di P3 riavvia P2, allora verrà eseguito un write(x) con x=0.

Esercizio 9)

Soluzione

```
semaphore cab = n, bas = k;
begin
        wait(cab);
        \\ change into swimwear;
        wait(bas);
        \\ fill basket;
        signal(cab);
        \\ swim;
        wait(cab);
        \\ empty the basket;
        signal(bas);
        \\ change into street clothes;
        signal(cab);
end
```

Il deadlock è possible. Un nuovo arrivato occupa uno spogliatoio, ma deve attendere che si liberi un cesto. Tuttavia, durante l'attesa, potrebbe bloccare le possibili uscite dall'impianto (perché chi vuole andarsene non ha dove rivestirsi) con la conseguente impossibilità ad ottenere cesti vuoti.

Esercizio 10)

Fornitore

```
do forever {
      wait(lock);
      randNum = rand( 1, 3 ); // Pick a random number from 1-3
      if ( randNum == 1 ) {
        // Put risorsa2, Put risorsa3
        signal(attuatore1);  // Wake up attuatore1 with risorsa1
      } else if ( randNum == 2 ) {
       // Put risorsa1, Put risorsa3
        signal(attuatore2);  // Wake up attuatore2 with risorsa2
      } else {
       // Put risorsa1, Put risorsa2
        signal(attuatore3); } // Wake up attuatore3 with risorsa3
        signal(lock);
        wait(agent);   //  Attuatore_i sleeps
} // end forever loop
```

Attuatore_i.

```
 do forever {
       wait(attuatore_i);  // Sleep right away
       wait(lock);
      // Pick up risorsa_i+1, Pick up risorsa_i+2
      signal(agent);
      signal(lock);
     // Act
}
```

Esercizio 11)

```
var     pieno: semaphore (initial 1);
        vuoto: semaphore (initial 0);
        index: integer;
        buff_msg: char[80];
        dim_msg: integer;

Procedure invio(x: char[80]; y: integer);
begin
        wait(pieno);
        buff_msg := x;
        dim_msg := y;
        index := 1;
        signal(vuoto)
end;
```

```
Procedure ricezione(var ch: char);
begin
        wait(vuoto);
        ch := buf_msg[index];
        index := index + 1;
        if index > dim_msg
          then signal(pieno)
          else signal(vuoto)
end;
```

Serve per continuare a leggere tutto il messaggio

Esercizio 12)

```
semaphore MUTEX = 1; semPRIV_D = 0; semPRIV_U = 0 ;
integer attesaU = 0, attesaD = 0, utilizzoD = 0, utilizzoU = 0;

RichiestaD
begin
        wait(MUTEX);
        if (utilizzoU=0) begin
                utilizzoD++;
                signal(semPRIV_D)
        end else
                attesaD++
        signal(MUTEX)
        wait(semPRIV_D)
end
```

```
RilascioD
begin
        wait(MUTEX);
        utilizzoD—;
        if (utilizzoD==0) begin
                while (attesaU>  0) begin
                        signal(semPRIV_U)
                        attesaU—
                        utilizzoU++
                end
        end
        signal(MUTEX)
        end
```

La stessa cosa, in maniera speculare, dovrà essere fatta per gli UOMINI.

Non ci sono uomini allora entro in bagno, se c'erano uomini aspettavano con il wait.

Non ci sono più donne entrano uomini

Dopo il wait(semPRIV_D) si intende donna entra in bagno. Dopo "if -else" ci sarebbe "qualcosa non specificato" che non conta nella concorrenza, tipo:
DonnaInBagnoFaQualcosa();

Dal testo avete, al tempo iniziale: P1 in esecuzione, P2 è pronto (dunque accede alla CPU appena questa si libera), P3 è sospeso nella coda di sem. Le code dei processi pronti e di quelli sospesi in un semaforo sono separate.
Nel punto 1) P1 si sospende in sem, dunque P2 che era unico processo pronto va in esecuzione
Nel punto 2) P2 lancia signal, P3 si sveglia (diventa processo pronto) e avendo priorità maggiore di P2 diventa processo in esecuzione.
Nel punto 3) P3 si sospende di nuovo, dunque finisce nella coda del semaforo; P2 torna in esecuzione
Nel punto 4) P2 lancia signal, P1 si sveglia però ha la stessa priorità di P2, dunque FIFO, P2 rimane in esecuzione.
PS
Signal fa uscire dalla coda di sem uno dei processi sospesi su tale semaforo.

Esercizio 13)

Sequenza di eventi	In Esecuzione	Coda Pronti	Valore di *sem*	Coda di *sem*
1) P1 esegue *wait (sem)*	P2	∅	0	P3 → P1
2) il processo in esecuzione esegue *signal(sem)*	P3	P2	0	P1
3) il processo in esecuzione esegue *wait (sem)*	P2	∅	0	P1 → P3
4) Il processo in esecuzione esegue *signal(sem)*	P2	P1	0	P3

La priorità si considera solo con i processi pronti, nella coda sem è mero FIFO

Esercizio 14)

utilizzo di rc: Il primo reader prende il controllo di db, poi solo Reader leggono, finché rc = 0, dunque non ci sono altri reader

1) Si possono avere corse critiche (interferenze) fra i lettori e gli scrittori e fra più scrittori;
2) I lettori possono accedere al database mentre uno scrittore lo sta modificando;
3) Gli scrittori possono rimanere bloccati per sempre in attesa su db.

Più scrittori scrivono sul file e lettori leggere Mentre ancora stanno scrivendo

Esercizio 15)

Soluzione

```
void studente (void)
{
        wait(fila);
        if(moduli == 0)
        {
           signal(campanello);
           wait(disp_moduli);
        }
        moduli —;
        signal(fila);
}
```

```
void impiegato(void)
{
        while(true)
        {
                wait(campanello);
                moduli = 50;
                signal(disp_moduli);
        }
}
```

Esercizio 16)

```
MONITOR cc
{
        int saldo=0, n-sosp=0;
        condition sosp;

        void prelievo(int cifra)
        {
            int presi = 0;  // presi è una variabile booleana
            do
            {
                if (saldo > cifra)
                {
                    saldo −= cifra;
                    presi = 1;
                }
                else
                {
                    n-sosp++;
                    sosp.wait();
                    if (n-sosp > 0)
                    {
                        n-sosp−−;
                        sosp.signal();
                    }
                }
            }while (!presi);
        }

        void deposito(int cifra)
        {
            saldo += cifra;
            if (n-sosp > 0)
            {
                n-sosp−−;
                sosp.signal();
            }
        }
}
```

Potete anche svegliarli tutti, rendete più lenta la procedura deposito; ma tanto, in tale esercizio, oltre che aggiornare cifra altro non esegue dopo aver svegliato i processi sospesi.

Esercizio 17)

Soluzione con semafori

```
Risorse condivise
  semaphore  mutex=1, pieno=0, vuoto=0;
  int porzioni=M;

Processo Cannibale
{
    ……
    wait(mutex);
    if (porzioni==0)
    {
        signal(vuoto);
        wait(pieno);
        porzioni=M;
    }
    porzioni−−;
    signal(mutex);
    <mangia porzione>
    ……
}

Processo Cuoco
{
    ……
    while(1)
    {
        wait(vuoto);
        <prepara le porzioni e riempi la pentola>
        signal(pieno);
    }
}
```

```
MONITOR SelvCena
{
        int pasti=0, vcannibale=0, vcuoco=0;  // vcuoco è una variabile booleana
        condition cannibale, cuoco;

        void serviti()
        {
            if (pasti == 0)
            {
                if (vcuoco)
                {
                    vcuoco = 0;
                    cuoco.signal();
                }
                else
                {
                    vcannibale++;
                    cannibale.wait();
                }
            }
            pasti−−;
        }

        void riempi()
        {
            if (pasti > 0)
            {
                vcuoco = 1;
                cuoco.wait();
            }
            pasti = M;
            while(vcannibale)
            {
                vcannibale−−;
                cannibale.signal();
                if (pasti == 0)
                    pasti = M;
            }
        }
}
```

Aspetta di essere servito

Serve i pasti ad oltranza

Esercizio 18)

```
int ita=0, fra=0;
semaphore mutex1=1, mutex2=1, piazza=1;

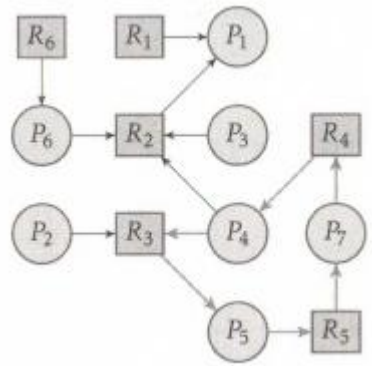
void italiano()
{
       wait(mutex1);
          ita++;
          if(ita==1) wait(piazza);
       signal(mutex1);
       …
       attraversa();
       …
       wait(mutex1);
          ita--;
          if(ita==0) signal(piazza);
       signal(mutex1);
}
```

Esercizio 19)

```
  semaphore first=0, second=0;
  process Giulio{
    while (true) {
      // Giulio prepara il panino //
      wait(first);
      signal(second);
      // Giulio consegna il panino //
    }
  }

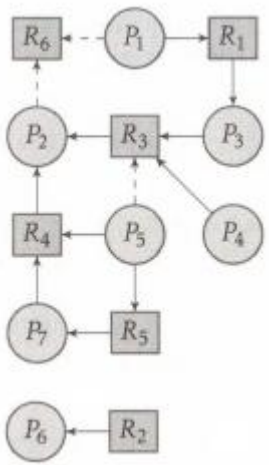

process Giovanni {
  while (true) {
    // Giovanni realizza il cocktail //
    signal(first);
    wait(second);
    // Giovanni consegna il cocktail //
  }
}
```

Esercizio 20)



ciclo $R_3 \rightarrow P_5 \rightarrow R_5 \rightarrow P_7 \rightarrow R_4 \rightarrow P_4 \rightarrow R_3$

Esercizio 21)



grafo non contiene cicli e dunque, avendo tutte le risorse una sola istanza, il sistema non può essere in *deadlock*.

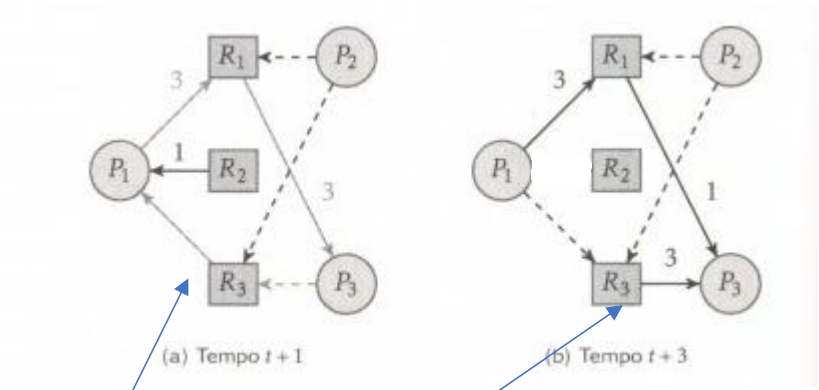
La richiesta di P_1 di accedere a R_6 non potrebbe essere soddisfatta all'istante t, nonsotante la risorsa sia disponibile ed esista il corrispondente arco di richiesta futura, in quanto convertendo quest'ultimo in arco di assegnamento si creerebbe nel grafo di allocaizone delle risorse il ciclo $P_1 \rightarrow R_1 \rightarrow P_3 \rightarrow R_3 \rightarrow P_2 \rightarrow R_6 \rightarrow P_1$.

La richiesta di P_5 di accedere a R_3 non potrebbe essere soddisfatta in quanto la risrosa risulta già assegnata al processo P_2.

La richiesta di P_1 di accedere a R_2 dovrebbe essere considerata invalida in quanto non esiste nel grafo di allocazione il corrispondente arco di richiesta futura.

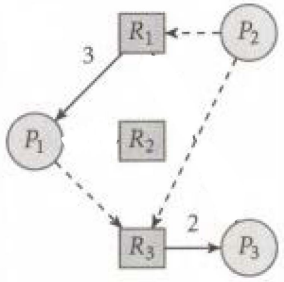
La richiesta di P_2 di accedere a R_6 verrebbe soddisfatta, in quanto la risorsa è libera, nel grafo esiste il corrispondente arco di richiesta futura, e convertendo quest'ultimo in arco di assegnamento non si vanno a creare cicli nel grafo di assegnamento delle risorse.

Esercizio 22)



allocando la risorsa si ottenga un ciclo nel grafo, evidenziato in grigio. Per questo motivo la richiesta non può essere soddisfatta e deve quindi essere lasciata in sospeso.

Al tempo $t+3$ la risorsa R_3 risulta pertanto libera; inoltre convertendo l'arco da P_3 a R_3 in un arco di assegnazione non si ottengo- cicli



(a) Tempo $t+4$

Al tempo $t+4$ il processo P_3 libera la risorsa R_1, che può dunque essere allocata a P_1, convertendo l'arco di richiesta in arco di allocazione

Al tempo $t+6$, dopo che il processo P_3 rilascia la risorsa R_3, è possibile riprendere in considerazione la richiesta lasciata in sospeso e soddisfarla;

Esercizio 23)

I numeri sono le priorità

Anche al tempo 1 C'è preemption

Tempo (*)		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
Stato dei Job	A	2							3						2						3				3	
	B		3																							
	C			2					3						3			3				4		4		
	D				2				3							4										
	E					2			3						4						4					

Istante di completamento: A 24, B 7, C 23, D 19, E 22

C cambia priorità dopo 5 istanti, è errato, deve sempre aspettare 6

Nota: La preemption avviene al tempo 13: C sostituisce A; e al tempo 15: D sostituisce C. Al tempo 19: la coda di priorità 4 ha E in testa e C in coda (C entra nella coda dopo E)

Esercizio 24)

Assumendo che si ordini rispetto agli indici in caso di ambiguità:
SJF: P4 P2 P5 P1 P6 P3
Priorità: P1 P2 P3 P4 P5 P6
RR: P1 P2 P3 P4 P5 P6 P1 P2 P3 P5 P6 P1 P3 P6 P3 (ognuno per 2 unità)

Esercizio 25)

1) FIFO: Sì
2) RR: Sì, con qualsiasi quanto di tempo di ampiezza maggiore o uguale al massimo tempo di esecuzione fra i processi considerati (ovvero 10, relativo a P3)

Esercizio 26)

A1(0,4) – A2(4,6) – B1(6,8) – A3(8,9) – B2(9,11) – A4(11,15) – B1(15,17) – B3(17,19) – B2(19,21) – B1(21,22) – B3(22,23) – C1(23,30) – C2(30,38) – C3(38,40)
A1, A3, A4 aspettano 0
A2 aspetta 2
B1 aspetta 15
B2 aspetta 13
B3 aspetta 11
C1 aspetta 20
C2 aspetta 25
C3 aspetta 31
T_m (attesa) = (2+15+13+11+20+25+31)/10 = 11,7 ms

B3 arriva dopo il quanto di tempo dato a B1, B3 viene messo nella coda dopo B1, dopo esecuzione di B2 avrò nella coda: B2->B3->B1, dunque eseguo B1, B3, B2 …

Esercizio 27)

al tempo t=19 si ha contemporaneamente la fine del quanto di tempo di P4 e il ritorno dal burst I/O di P2.
Tutte le volte che qualcosa accade "contemporaneamente" devi avere regola per gestire come mettere in coda i due processi, dipende dall'implementazione reale dell'algoritmo.

Soluzione 1

RR quanto 2

CPU: P1(0,2), P2(2,4), P3(4,6), P4(6,8), P2(8,9), P3(9,11), P4(11,13), P3(13,15), P1(15,17), P4(17,19), P3(19,20), P2(20, 22), P4(22,24), P2(24,25), P4(25,31), P3(31,33), P4(33,35), P3(35,37), P4(37,39), P3(39,42), VUOTO(42,49), P4(49,67)

I/O: P1(2,12), P2(9,19), P3(20,30), P4(39,49)

TA = (T(P1)+T(P2)+T(P3)+T(P4))/4 = (3+9+18+21)/4 = 12,75

SJF preemptive

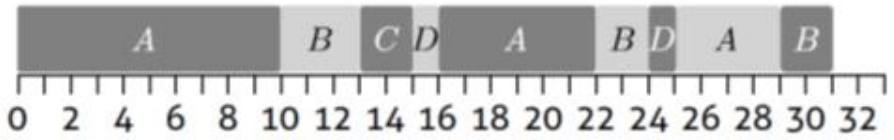
CPU: P1(0,2), P2(2,5), P3(5,12), P1(12,14), P4(14,15), P2(15,18), P4(18,22), P3(22,29), P4(29,42), VUOTO(42,52), P4(52,70)

I/O: P1(2,12), P2(5,15), P3(12,22), P4(42,52)

TA = (T(P1)+T(P2)+T(P3)+T(P4))/4 = (0+2+5+24)/4 = 7,75

Soluzione 2

Il diagramma di Gantt è:



Tempi di attesa e di risposta:

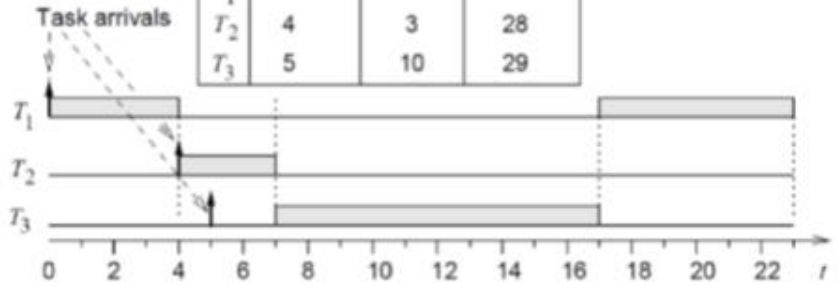
$$w_A = (0-0)+(16-10)+(25-22) = 9$$
$$w_B = (10-2)+(22-13)+(29-24) = 22$$
$$w_C = 13-3 = 10$$
$$w_D = (15-5)+(24-16) = 18$$

Sì, questo scheduling soffre dell'effetto convoglio dato che i CPU–burst di A sono sempre significativamente più lunghi rispetto a quelli degli altri processi.

Soluzione 3

Il diagramma di Gantt è:

	arrival	duration	deadline
T_1	0	10	33
T_2	4	3	28
T_3	5	10	29



Esercizio 28)

La politica di scheduling utilizzata effettua la prelazione della CPU ai processi in esecuzione, in quanto all'istante t6 il processo P1, che ancora non ha terminato la propria esecuzione, viene ripor-tato nella coda dei processi pronti al fine di eseguire il processo P2.

Nome	Tempo di arrivo	Tempo di terminazione	Tempo di *turnaround*
P_0	0	15	15
P_1	4	12	8
P_2	6	10	4
P_3	11	18	7
P_4	15	20	5

- P_1 ha priorità maggiore di P_0;
- P_2 ha priorità maggiore di P_1;
- P_3 ha priorità minore di P_1;
- P_4 ha priorità minore di P_3.

Tempo medio: (15+8+4+7+5)/5=7,8

P3 priorità minore di P0

Esercizio 29)

- Il tempo medio di *turnaround* è

$$\frac{3+8+10+16+23+26}{6} \approx 14.3$$

unità di tempo.

- Il tempo medio di attesa in coda è

$$\frac{8+10+16+23+3}{6} \approx 10$$

unità di tempo.



Il calcolo del numero medio di processi in coda è leggermente più laborioso: si nota che nei primi tre istanti di tempo in coda vi sono cinque processi (da P_1 a P_5), nei successivi cinque istanti ve ve sono quattro, nei successivi due se ne trovano tre, e così via. Per cui il numero medio di processi in coda sarà pari a

$$\frac{5\cdot 3+4\cdot 5+3\cdot 2+2\cdot 6+1\cdot 7}{26} = \frac{60}{26} \approx 2.31.$$

Come cambierebbero le risposte all'esercizio precedente se il sistema opera-tivo allocasse la CPU ai processi pronti utilizzando lo scheduling per brevità, assumendo che a parità di durata abbia priorità il processo P con il minore valore dell'indice i?

Siccome nello scheduling per brevità si alloca la CPU ai processi con minor tempo di CPU burst, i processi verranno eseguiti nel seguente ordine: P2, P0, P5, P1, P3 e P4

- Il tempo medio di *turnaround* è

$$\frac{2+5+8+13+19+26}{6} \approx 12.17$$

unità di tempo.

- Il tempo medio di attesa in coda è

$$\frac{2+5+8+13+19}{6} \approx 7.83$$

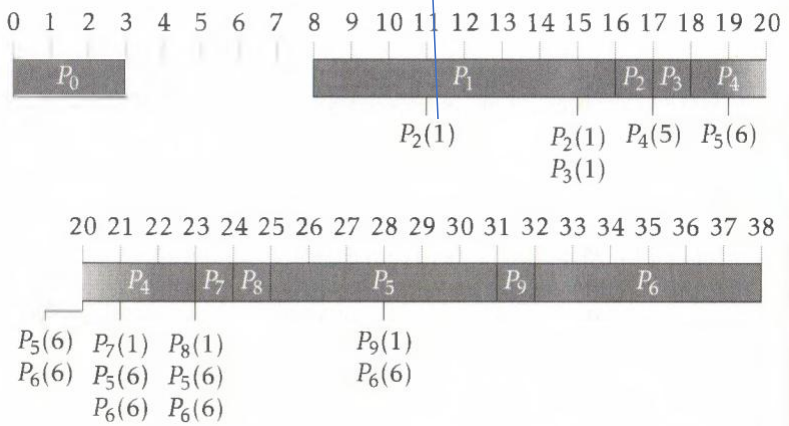
unità di tempo.

- Per quanto riguarda il calcolo del numero medio di processi in coda, si nota che nei primi tre istanti di tempo in coda vi sono cinque processi (da P_1 a P_5), nei successivi cinque istanti ve ve sono quattro, nei successivi due se ne trovano tre, e così via. Per cui il numero medio di processi in coda nel lasso di tempo considerato sarà pari a

$$\frac{5\cdot 2+4\cdot 3+3\cdot 3+2\cdot 5+1\cdot 6}{26} = \frac{47}{26} \approx 1.81.$$

Esercizio 30)

Id del processo in coda e fra () i secondi necessari per la sua Completa esecuzione

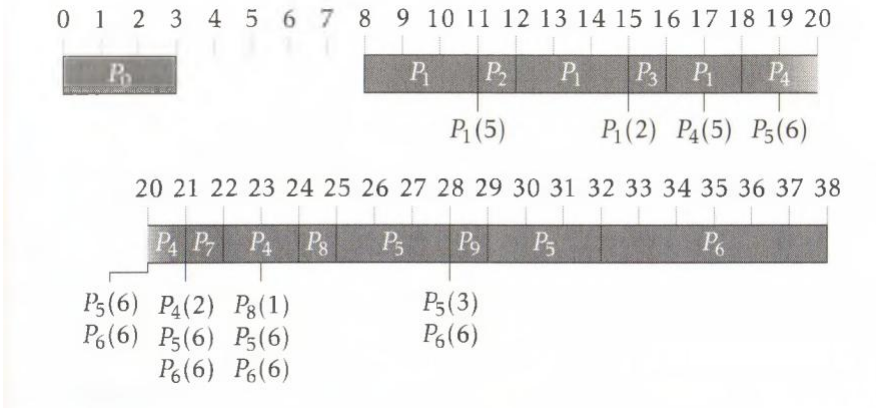


Nome	Tempo di arrivo	Tempo di terminazione	Tempo di *turnaround*
P_0	0	3	3
P_1	8	16	8
P_2	11	17	6
P_3	15	18	3
P_4	17	23	6
P_5	19	31	12
P_6	20	38	18
P_7	21	24	3
P_8	23	25	2
P_9	28	32	4

$$\frac{3+8+6+3+6+12+18+3+2+4}{10} = 6.5$$

unità di tempo.

Come cambierebbero le risposte all'esercizio precedente se il sistema opera-tivo prevedesse la prelazione?

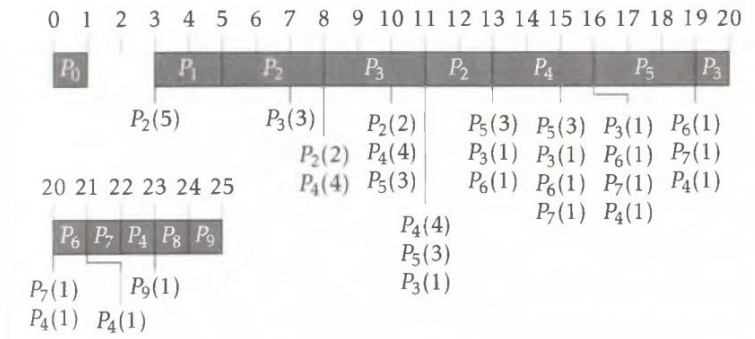


Nome	Tempo di arrivo	Tempo di terminazione	Tempo di *turnaround*
P_0	0	3	3
P_1	8	18	10
P_2	11	12	1
P_3	15	16	1
P_4	17	24	7
P_5	19	32	13
P_6	20	38	18
P_7	21	22	1
P_8	23	25	2
P_9	28	29	1

$$\frac{3+10+1+1+7+13+18+1+2+1}{10} = 5.7$$

Esercizio 31)

Nome	Tempo di arrivo	Durata
P_0	0	1
P_1	3	2
P_2	3	5
P_3	7	4
P_4	8	4
P_5	10	3
P_6	13	1
P_7	15	1
P_8	23	1
P_9	23	1



Nome	Tempo di arrivo	Tempo di terminazione	Tempo di *turnaround*	Tempo di attesa
P_0	0	1	1	0
P_1	3	5	2	0
P_2	3	13	10	5
P_3	7	20	13	9
P_4	8	23	15	11
P_5	10	19	9	6
P_6	13	21	8	7
P_7	15	22	7	6
P_8	23	24	1	0
P_9	23	25	2	1

Ist.	Processi in coda	Ist.	Processi in coda	Ist.	Processi in coda
0	0	8	2	16	4
1	0	9	2	17	4
2	0	10	3	18	4
3	1	11	3	19	3
4	1	12	3	20	2
5	0	13	3	21	1
6	0	14	3	22	0
7	1	15	4	23	1
				24	0

$$\frac{1+2+10+13+15+9+8+7+1+2}{10} = 6.8$$

Per rispondere all'ultimo quesito è necessario conteggiare, istan-te per istante, la lunghezza della coda dei processi in attesa. Calcolando la media aritmetica del numero di processi in coda si ottiene:

$$\frac{1\cdot 5+2\cdot 3+3\cdot 6+4\cdot 4}{25} = \frac{45}{25} = 1.8.$$

Esercizio 32)

An operating system supports a paged virtual memory, using a central processor with a cycle time of 1 microsecond. It costs an additional 1 microsecond to access a page other than the current one. Pages have 1000 words, and the paging device is a drum that rotates at 3000 revolutions per minute and transfers 1 million words per second. The following statistical measurements were obtained from the system:

- 1 percent of all instructions executed accessed a page other than the current page.
- Of the instructions that accessed another page, 80 percent accessed a page already in memory.
- When a new page was required, the replaced page was modified 50 percent of the time.

Calculate the effective instruction time on this system, assuming that the system is running one process only and that the processor is idle during drum transfers.

Tempo di accesso alla memoria secondaria: 10000 microsecondi per pagina. Dunque per trasferire 1000 words (i.e. one page): 1000 microsecondi. N.B. Per essere precisi si dovrebbe anche sommare il microsecondo del cycle time.

Answer:

$$\begin{aligned}\text{effective access time} &= 0.99 \times 1\ \mu\text{sec} + 0.008 \times (2\ \mu\text{sec}) \\ &\quad + 0.002 \times (10{,}000\ \mu\text{sec} + 1{,}000\ \mu\text{sec}) \\ &\quad + 0.001 \times (10{,}000\ \mu\text{sec} + 1{,}000\ \mu\text{sec}) \\ &= (0.99 + 0.016 + 22.0 + 11.0)\ \mu\text{sec} \\ &= 34.0\ \mu\text{sec}\end{aligned}$$

Esercizio 33)

You have devised a new page-replacement algorithm that you think may be optimal. In some contorted test cases, Belady's anomaly occurs. Is the new algorithm optimal? Explain your answer.

Answer:

No. An optimal algorithm will not suffer from Belady's anomaly because —by definition—an optimal algorithm replaces the page that will not be used for the longest time. Belady's anomaly occurs when a page-replacement algorithm evicts a page that will be needed in the immediate future. An optimal algorithm would not have selected such a page.

Esercizio 34)

Si consideri un processo con 7 pagine logiche di 1K, e la seguente configurazione della tabella delle pagine (1=sì, 0=no):

Pag. Logica	Presente/Assente	Riferita	Modificata	Pag. Fisica
0	1	1	0	512
1	0	0	0	0
2	1	0	0	102
3	0	0	0	0
4	1	0	0	428
5	1	1	0	123
6	1	1	0	812

e della TLB (memoria associativa) di 4 posizioni

Posizione	Valida	Modificata	Pag. Virtuale	Pag. Fisica
0	1	0	5	123
1	1	1	6	812
2	0	0	0	0
3	1	0	0	512

Discutere cosa accade e mostrare come viene modificato il contenuto della TLB e della tabella delle pagine se il processo accede in scrittura alle locazioni 4000 e 5130.

- 4000: Si accede alla pagina logica 3 che non è presente né nella TLB né in memoria, verrà pertanto generato un page fault. Successivamente il corrispondente descrittore di pagina verrà caricato nella tabella delle pagine, marcato come riferito, e caricato nella TLB. A questo punto nella TLB la pagina viene marcata come modificata. Quindi la tabella delle pagine in corrispondenza della pag. fisica 3 verrà modificata come segue:

3	1	1	1	XXX

mentre la TLB, nella riga 2, verrà modificata nel modo seguente:

2	1	1	3	XXX

Riga 2 è unica libera

- 5130: Si accede alla pagina logica 5 il cui descrittore si trova nella TLB, quindi viene tradotto l'indirizzo usando le informazioni presenti nella TLB e nella TLB la pagina viene marcata come modificata. Quindi la TLB, nella riga 0 verrà modificata nel modo seguente:

0	1	1	5	123

Non abbiamo informazioni su dove verrà caricata la pagina fisica

PS anche tabella della pagine dovrà avere bit di modifica a 1 per tale pagina logica

Esercizio 35)

$t = 0$ Quando viene eseguito P_1, l'intera zona di memoria dedicata ai processi è libera, quindi il processo viene allocato nelle celle i cui indirizzi vanno da 0 a 1999, lasciando libera l'area che va dall'indirizzo 2000 all'indirizzo 4095.

$t = 2$ Anche quando viene eseguito il processo P_2 nel sistema è presente una sola area di memoria libera (quella descritta al punto precedente): l'allocazione viene dunque fatta a partire dall'indirizzo 2000.

$t = 3$ Per P_3 valgono le stesse considerazioni dei punti precedenti: il processo viene allocato nella parte iniziale dell'unica area di memoria libera.

$t = 4$ Quando il processo P_1 termina, la memoria che occupava viene rilasciata e dunque nel sistema vi sono due aree libere, che raggruppano rispettivamente i primi 2000 e gli ultimi 586 byte.



(a) $t = 0$



(b) $t = 2$



(c) $t = 3$



(d) $t = 4$



(e) $t = 6$

$t = 6$ Siccome entrambe le aree libere descritte al punto precedente sono più grandi di 580 byte, l'algoritmo *first fit* alloca il processo P_4 nella prima di queste due aree.

$t = 7$ Dopo che il processo P_4 è stato caricato in memoria, vi saranno due aree libere grandi rispettivamente 1420 e 586 byte: siccome P_5 ha dimensioni maggiori di entrambe queste aree, il processo non potrà essere caricato.

Esercizio 36)



(a) $t = 0$



(b) $t = 7$



(c) $t = 10$



(d) $t = 11$



(e) $t = 13$

$t = 0$ Quando viene eseguito il primo processo, l'intera memoria è libera e dunque vi è un'unica area contigua che va dall'indirizzo 0 all'indirizzo 4095. Quest'area verrà divisa in due parti: una che va dall'indirizzo 0 all'indirizzo 2047, in cui allocare il processo P_1, e una di memoria libera, dall'indirizzo 2048 all'indirizzo 4095.

$t = 7$ Quando viene eseguito il secondo processo, P_1 è ancora in esecuzione, dunque vi sarà un'unica zona di memoria libera che va dall'indirizzo 2048 all'indirizzo 4095: P_2 verrà dunque allocato a partire dall'indirizzo 2048, e risulterà libera la memoria che va dall'indirizzo 4018 all'indirizzo 4095.

$t = 10$ Quando P_1 termina, la memoria che occupava viene rilasciata. Ci saranno quindi due zone libere: L_1, grande 2 Kbyte e posizionata a partire dall'indirizzo 0, e L_2, grande 38 byte e posizionata nelle ultime celle di memoria.

$t = 11$ Il processo P_3 deve essere necessariamente allocato all'interno dell'area L_1 descritta al punto precedente, in quanto l'area libera rimanente è grande 38 byte, mentre P_3 necessita di 2000 byte. Pertanto P_3 sarà allocato tra gli indirizzi 0 e 1999, e lo spazio rimanente, pari a 48 byte, andrà a formare una nuova area libera L_3.

$t = 13$ Quando viene eseguito il processo P_4, in memoria ci sono le due zone libere L_2 e L_3 descritte al punto precedente, entrambe abbastanza capienti per poter accogliere il processo. Siccome il sistema utilizza l'algoritmo *worst fit*, verrà scelta l'area L_3 in quanto è quella più grande.

Infine, siccome all'istante $t = 13$ nel sistema sono presenti due aree libere, grandi rispettivamente 48 e 48 byte, si avrebbe frammentazione esterna nel caso in cui dovesse venire eseguito un processo che richiede uno spazio di memoria tra 49 e 96 byte. Non vi può invece essere frammentazione interna in quanto a ogni processo viene allocato esattamente lo spazio in memoria di cui ha bisogno.

Esercizio 37)

Si consideri un sistema operativo in cui la memoria centrale è allocata in modo distribuito ai processi tramite la tecnica della paginazione, utilizzando nel contampo tecniche di memoria virtuale. Si assuma che la memoria fisica sia divisa in *frame* di un Kbyte, che ogni indirizzo di memoria fisica occupi 32 bit e che in un indirizzo logico i bit più significativi indichino il numero di pagina e quelli meno significativi indichino lo scostamento.

1. Quanti bit sono necessari per specificare lo scostamento all'interno di una pagina?
2. Ipotizzando che anche l'indirizzo di una cella di memoria occupi 32 bit, qual è la quantità massima di memoria indirizzabile?

Si consideri la tabella delle pagine di processo, de-scritta nella Tabella in cui il bit di validità è impostato a 1 per le pagine già caricate in memoria centrale:

N. di pagina	bit di validità	N. di *frame*
0	1	739
1	1	127
2	0	–
3	1	91
4	0	–
5	1	470
6	1	2
7	1	106
8	1	495
9	0	–

Specificare che cosa succede quando il processo tenta di accedere ai seguenti indirizzi logici (espressi in notazione binaria, dividendo i bit in quartetti e abbreviando gli zero iniziali, al fine di facilitare la leggibilità), indicando ove possibile quali siano gli indirizzi fisici corrispondenti, esprimendoli in notazione decimale:

(a) $(0\ldots0\,0111\ 1101\ 0101)_2$,

(b) $(0\ldots0\,0100\ 0001\ 0111)_2$,

(c) $(0\ldots0\,1011\ 1101\ 1011)_2$.

1. Siccome una cella di memoria occupa 32 bit, che corrispondono a 4 byte, e la grandezza delle pagine e dei *frame* è di un Kbyte che corrisponde a 1024 byte, all'interno di pagine e *frame* ci saranno $1024/4 = 256 = 2^8$ diverse celle. Pertanto saranno necessari 8 bit nell'indirizzo per poter specificare tutti i valori possibili per lo scostamento.

2. Dal punto precedente si ottiene che dei 32 bit che compongono l'indirizzo di una cella, 8 vanno dedicati allo scostamento. Quindi i 24 bit rimanenti possono essere utilizzati per specificare il numero di pagina. Pertanto in linea di principio è possibile avere dei processi con 2^{24} pagine differenti, per un totale di $2^{24} \cdot 1024 = 2^{34} = 16 \cdot 2^{30}$ byte, cioè 16 Gbyte.

3. Considerando gli indirizzi logici indicati, si verificano i seguenti casi:

   (a) Dividendo l'indirizzo logico $(0\ldots0\ 0111\ 1101\ 0101)_2$ nelle parti che comprendono i 24 bit più significativi e gli otto meno significativi, eliminando gli zero iniziali si ottiene $(111)_2 = 7$ in corrispondenza del numero di pagina e $(1101\ 0101)_2 = 213$ per quanto riguarda lo scostamento. Dalla tabella delle pagine si verifica che la pagina 7 è allocata nel *frame* 106, per cui l'indirizzo fisico a cui si accederà sarà

   ```
   106 * 256 + 213 = 27349 cella
   ```

   (b) L'indirizzo logico $(0\ldots0\ 0100\ 0001\ 0111)_2$ si riferisce alla pagina $(100)_2 = 4$, ma in corrispondenza di quest'ultima si verifica che la tabella delle pagine ha il bit di validità pari a 0. Pertanto l'accesso a questo indirizzo logico genera un'assenza di pagina che andrà gestita dal *pager*.

   (c) Analizzando l'indirizzo logico $(0\ldots0\ 1011\ 1101\ 1011)_2$ si verifica come la pagina a cui questo fa riferimento sia $(1011)_2 = 11$, il cui numero non compare nella tabella delle pagine: l'accesso è quindi invalido e il gestore della memoria genererà una *trap*.

Esercizio 38)

Si consideri un sistema operativo dotato di memoria virtuale con paginazione a richiesta pura. Si presupponga di eseguire un processo che richieda, nell'ordine, di accedere alle seguenti pagine:

$$2, 6, 3, 2, 9, 10, 9, 3, 11, 12, 3, 2.$$

Avendo a disposizione un massimo di 4 *frame* a cui associare le pagine del processo, si esegua l'algoritmo FIFO al fine di calcolare il numero di *page fault* che si verificano durante l'esecuzione del processo.

$t = 1, 2, 3$ Quando il processo viene eseguito, nessuna delle sue pagine è presente in memoria, quindi i primi tre accessi, tutti riferiti a pagine diverse, generano dei *page fault*.

$t = 4$ La pagina 2 è già presente in memoria, quindi non c'è assenza di pagina e il contenuto dei *frame* non cambia.

$t = 5, 6$ I successivi due accessi riguardano pagine non caricate in memoria centrale, quindi causano due *page fault*. La pagina 9 viene inserita nell'unico *frame* libero, mentre la pagina 10 richiede un rimpiazzamento: l'algoritmo FIFO selezionerà quindi il *frame* contenente la pagina presente da più tempo in memoria centrale, cioè la pagina 2.

$t = 7, 8$ Gli accessi alle pagine 9 e 3 non causano assenze di pagina.

$t = 9, \ldots, 12$ Gli ultimi quattro accessi alla memoria generano dei *page fault*, così che le pagine cui si accede vanno a rimpiazzare rispettivamente le pagine 6, 3, 9 e 10.

t	1	2	3	4	5	6	7	8	9	10	11	12
Pagina	2	6	3	2	9	10	9	3	11	12	3	2
	2	6	3	3	9	10	10	10	11	12	3	2
	–	2	6	6	3	9	9	9	10	11	12	3
	–	–	2	2	6	3	3	3	9	10	11	12
	–	–	–	–	2	6	6	6	3	9	10	11

Esercizio 39)

Si consideri un sistema operativo dotato di memoria virtuale con paginazione a richiesta pura e rimpiazzamento con algoritmo LRU (*least recently used*). Si presupponga di eseguire un processo che richieda, nell'ordine, di accedere alle seguenti pagine:

8, 0, 10, 12, 9, 10, 11, 3, 11, 11.

Avendo a disposizione 3 *frame* a cui associare le pagine del processo, si calcoli il numero di *page fault* che si verificano durante l'esecuzione del processo.

t	1	2	3	4	5	6	7	8	9	10
Pagina	8	0	10	12	9	10	11	3	11	11
	8	0	10	12	9	10	11	3	11	11
	–	8	0	10	12	9	10	11	3	3
	–	–	8	0	10	12	9	10	10	10

$t = 1,2,3$ Quando il processo effettua i suoi primi tre accessi in memoria, i *frame* non contengono alcuna pagina, pertanto gli accessi generano ciascuno un *page fault* e vanno a riempire i tre *frame*.

$t = 4,5$ I due successivi accessi riguardano le pagine 12 e 9, entrambe assenti, e quindi generano dei *page fault*. Tali pagine andranno a rimpiazzare rispettivamente le pagine 8 e 0, a cui si è acceduto meno recentemente.

$t = 6$ La pagina 10 è già in memoria, pertanto non vi sono *page fault* e l'occupazione dei *frame* resta invariata.

$t = 7,8$ Gli accessi alle pagine 11 e 3 generano *page fault*, i quali a loro volta causano il rimpiazzamento delle pagine 12 e 9.

$t = 9,10$ La pagina 11 è già in memoria centrale, dunque gli ultimi due accessi alla memoria non generano *page fault*.

Esercizio 40)

Si consideri un sistema operativo dotato di memoria virtuale con paginazione a richiesta pura e rimpiazzamento con algoritmo LRU (*least recently used*), abbinato a uno schema di prepaginazione che carica le prime pagine di un processo fino a riempire i *frame* disponibili. Si presupponga di eseguire un processo che richieda, nell'ordine, di accedere alle seguenti pagine:

15, 15, 0, 2, 3, 11, 2, 6, 4, 14.

Avendo a disposizione 4 *frame* $f_0, \ldots, f_3$ cui associare le pagine del processo, e ipotizzando che quando vi siano più *frame* in cui inserire una pagina questa venga assegnata al *frame* f_i con il minimo indice possibile, si calcoli il numero di *page fault* che si verificano durante l'esecuzione del processo.

t	0	1	2	3	4	5	6	7	8	9	10
Pagina		15	15	0	2	3	11	2	6	4	14
f_0	0	15	15	15	15	15	11	11	11	11	14
f_1	1	1	1	0	0	0	0	0	6	6	6
f_2	2	2	2	2	2	2	2	2	2	2	2
f_3	3	3	3	3	3	3	3	3	3	4	4

$t = 1$ La pagina 15 non si trova in memoria centrale, e tutti i *frame* contengono pagine a cui non si è mai acceduto. La pagina verrà quindi allocata nel *frame* f_0 in quanto il suo indice è quello più piccolo.

$t = 2$ La pagina a cui si accede è già caricata in memoria centrale, quindi non vi sono variazioni.

$t = 3$ La pagina 0 non è inserita in memoria centrale, e solo il *frame* f_0 contiene una pagina a cui si è precedentemente acceduto. Potendo scegliere tra i *frame* f_1, f_2 e f_3, la pagina verrà caricata in f_1 in quanto il suo indice è quello più basso.

$t = 4,5$ Le pagine 2 e 3 si trovano già in memoria centrale, quindi non vi sono variazioni.

$t = 6$ La pagina 11 genera un *page fault*, e tra quelle caricate in memoria centrale quella acceduta meno recentemente è la pagina 15, che verrà quindi rimpiazzata dalla pagina 11.

$t = 7$ La pagina 2 è già presente in memoria.

$t = 8,9,10$ Le pagine 6, 4 e 14 non sono presenti in memoria centrale, e l'algoritmo LRU le sostituisce rispettivamente alle pagine 0, 3 e 11.

Esercizio 41)

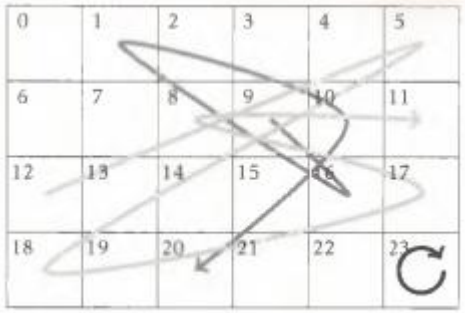
0:		1:	10	2:		3:		4:		5:	18
6:		7:		8:	11	9:	16	10:	20	11:	−1
12:	5	13:		14:		15:		16:	1	17:	8
18:	17	19:		20:	−1	21:		22:		23:	23

ogni riquadro rappresenta un blocco da 4 Kbyte, a sinistra del quale è riportato in grigio il corrispondente indirizzo, e in ogni blocco viene ri-servato un byte per gestire il puntatore al prossimo blocco. Il contenuto
di ogni riquadro può essere:

- un numero non negativo *n*, a indicare che il blocco è occupato dai contenuti di un file che prosegue nel blocco numero *n*
- il numero - 1 , quando il blocco contiene la parte finale di un file;
- nulla, nel caso in cui il blocco sia libero.

Ipotizzando che su questo disco sia memorizzato un intero file System che contiene due file, memorizzati rispettivamente a partire dai blocchi con indirizzo 9 e 12, si risponda alle seguenti domande:

1. La rappresentazione dei blocchi del disco contiene degli errori? Perché?
2. Assumendo che lo spazio libero sul disco sia gestito utilizzando un vettore di bit (*bitmap*) in cui a ogni blocco è associato un bit il cui valore è pari a 1 se il blocco risulta libero e pari a 0 altrimenti, quale sarà il contenuto della *bitmap*?
3. Ipotizzando che lo spazio libero venga allocato scandendo la *bitmap* da sinistra verso destra, in quali blocchi verrebbe memorizzato un nuovo file lungo 21547 byte?



necessario controllare anche che non vi siano blocchi che formalmente risultano occupati ma che non fanno parte di alcun file. Si verifica facilmente che questo è ciò che accade esattamente con il blocco avente indirizzo 23, che oltretutto è indicato come parte di un file i cui contenuti continuano nel blocco stesso.

2. Se si costruisce la *bitmap* scorrendo i blocchi per indirizzo crescente e associando i blocchi liberi al valore 1 e quelli occupati al valore 0, si ottiene la seguente rappresentazione:

   101110110000011100010110.

3. All'interno dei 4 Kbyte contenuti in ogni blocco, uno è destinato a contenere il puntatore al blocco successivo o il valore speciale che indica la fine del file, in ogni blocco saranno effettivamente a disposizione 4095 byte. Quindi un file di 21547 richiederà $\lceil 21547/4095 \rceil = 6$ blocchi liberi. Scandendo la *bitmap* da sinistra verso destra, tale file verrà pertanto memorizzato nei blocchi aventi indirizzo 0, 2, 3, 4, 6 e 7.

1 KB * 4 = (4*1024 Bytes)

Esercizio 42)

Siccome nella FAT vi sono 7 righe in cui compare il valore −1, la dimensione massima per un nuovo file corrisponde a 7 blocchi, che equivale a 7 · 4 KByte = 28KByte.

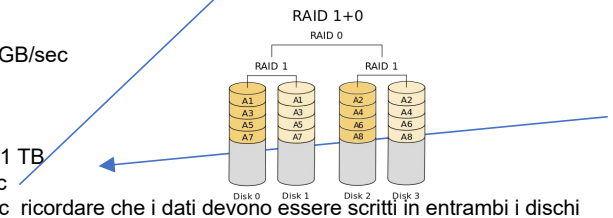
0	−1
1	6
2	−1
3	−1
4	11
5	9
6	15
7	−2
8	−1
9	19
10	−2
11	7
12	13
13	1
14	−1
15	10
16	−1
17	−1
18	12
19	4

Esercizio 43)

possibile anche con raid 0+1 la lettura parallela su 4 dischi

RAID 1
Capacità= 200 GB
Forte aumento tolleranza guasti (MTBF) . Es : se la probabilità di un guasto su un disco nell'arco delle prossime 24 ore è p,
la probabilità che nelle prossime 24 ore si guastino entrambi i dischi è p^2 (molto più bassa). Il disco guasto va comunque sostituito al più presto.
RAID0
Capacità = 400GB
Diminuzione tolleranza ai guasti: la probabilità che si guasti il sistema aumenta. Es.: se la probabilità di un guasto su un disco nell'arco delle prossime 24 ore è p, avendo due dischi la probabilità è 1-(1-p)*(1-p). Con p=0,0001 la probabilità diventa 0,00019999 (quasi il doppio).

RAID 0:
Ctot= 4*500 GB = 2000 GB = 2 TB
TRL = TRS = 120 *4 MB/sec = 480 GB/sec
Guasto di un disco: perdita dati
Guasto 2 dischi: perdita dati
RAID 10:
Ctot= (4 / 2) * 500 GB = 1000 GB = 1 TB
TRL = 120 *4 MB/sec = 480 GB/sec
TRS = 120 *2 MB/sec = 240 GB/sec ricordare che i dati devono essere scritti in entrambi i dischi
Guasto di un disco: no perdita dati, rallentamento solo in lettura, nessun rallentamento in scrittura
Guasto 2 dischi: perdita dati solo se i due dischi guasti erano in RAID1, altrimenti come sopra



abbiamo 4 dischi da 500, se ho due coppie in RAID1 allora la capacità totale sarà (4*500)/2, divido per 2 perché nel RAID1 un disco è mirror dell'altro.
TRL, ogni disco 120 MB/s, avendo 4 dischi dai quali posso leggere faccio 120*4.
TRS, devi ricordarti che ho due coppie in RAID1, dunque ho 2 dischi che sono mirror, per scrivere considero solo la banda di due dischi, gli altri due semplicemente copieranno quello che stai scrivendo.

Esercizio 44)

Sequenza di eventi	In Esecuzione	Coda Pronti	Valore di *sem*	Coda di *sem*
1) P1 esegue *P (sem)*	P2	∅	0	P3 ◊ P1
2) il processo in esecuzione esegue *V(sem)*	P3	P2	0	P1
3) il processo in esecuzione esegue *P (sem)*	P2	∅	0	P1 ◊ P3
4) Il processo in esecuzione esegue *V(sem)*	P2	P1	0	P3

Esercizio 45)

```
A (produttore):
........
While (true) {
      item = produce_item();
      P(&CelleLibere);
      P(mutex);
      insert(item);
      V(mutex);
      V(&MessaggiGiacenti);
      }
........

B (consumatore):
........
While (true) {
      P(&MessaggiGiacenti)
      P(mutex);
      item= remove_item();
      V(mutex);
      V(&CelleLibere);
      consume(item);
      }
........
```

Esercizio 46)

Nel problema dei processi lettori/scrittori può avvenire il fenomeno di attesa indefinita; infatti, se mentre è presente un processo lettore nella sezione critica, continuano ad arrivare altri lettori, tutti i processi scrittori dovranno aspettare (attesa indefinita) l'uscita dell'ultimo lettore dalla sua sezione critica, rilasciando così la mutua esclusione sul semaforo **db**.

```
semaphore mutex=1, db=1;
int rc=0;
void Lettore()
                {
                  wait(mutex);
                  rc++;
                  if (rc == 1)
wait(db);
                  signal(mutex);
                  LetturaDB();
                  wait(mutex);
                  rc--;
                  if (rc == 0)
signal(db);
                  signal(mutex);
                }
void Scrittore()
                {
                  wait(db);
                  ScritturaDB();
                  signal(db);
                }
```

Esercizio 47)

```
sem cont[M] = 1  // 1 semaforo per contenitore. Tutti inizializzati a 1
int liberi[M]  = K // array di interi condiviso. Tutti inizializzati a K

void cameriere( )
i = -1
while ( 1 )
   i = ( i+1 ) %M
   wait (cont[ i ])
        if (liberi [i] == 0)
                liberi [ i ] = K
                lava ( )
        endif
   signal (cont[ i ])
endwhile
end cameriere
```

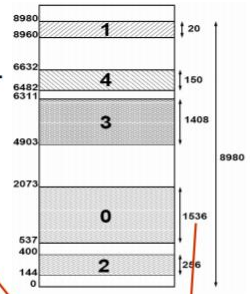
```
void studente ( )
i = -1
fatto == no
while ( fatto == no )
    i = ( i+1 ) %M
    wait (cont[ i ] )
        if (liberi [i] > 0)
                liberi [ i ] = liberi [ i ] - 1
                posa ( )
                fatto = si
        endif
    signal (cont[ i ] )
endwhile
end studente
```

Esercizio 48)

6 page fault

Esercizio 49)

Ricordando che ...



Allora:

indirizzo virtuale	indirizzo fisico
$< 0, 1984 >$	Errore: indirizzo illegale.
$< 1, 18 >$	$8960 + 0018 = 8978$
$< 2, 250 >$	$0144 + 0250 = 0394$
$< 3, 1400 >$	$4903 + 1400 = 6303$
$< 4, 112 >$	$6482 + 0112 = 6594$

1984 > 1536

Esercizio 50)

A ogni iterazione del programma la memoria viene acceduta per leggere l'istruzione successiva
→ la pagina relativa al processo verrà continuamente acceduta

I primi 200 azzeramenti faranno accesso a Pagina 1, i successivi 200 a Pagina 2; l'ulteriore azzeramento (cella `A[4][0]`) causerà un *page fault* ; la pagina usata meno recentemente è Pagina 1 che verrà sostituita con le celle da `A[4][0]` a `A[5][99]` incluse. Arrivati all'azzeramento di `A[6][0]` sarà Pagina 2 ad essere stata usata meno di recente
E così via, causando in tutto 1 *page fault* iniziale per caricare il processo in Pagina 0 e 50 *page fault* di Pagina 1 e 2 (25 ciascuno) per caricare le porzioni di matrice.

TOTALE = 51 *page fault*

Esercizio 51)

Caso a) La pagina C5 è già in memoria (nella pagina fisica 2) e quindi viene marcata come riferita

La lista risultante è dunque:
C2(15,1) - D8(27,0) - B5(30,1) - A1(37,0) - A0(50,1) - C5(70,1) - C9(92,0) - B3(97,0)

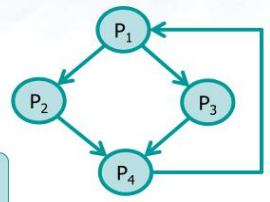
Caso b) La pagina logica A9 non si trova in memoria, l'algoritmo *second chance* esamina C2: siccome è stata riferita, viene inserita in coda alla lista ed il campo riferita viene posto uguale a 0; dopodichè, si esamina D8, che non risulta riferita e viene dunque eliminata. La lista risultante è dunque:

B5(30,1) - A1(37,0) - A0(50,1) - C5(70,1) - C9(92,0) - B3(97,0) - C2(15,0) - A9(105,1)

N.B a C2 accedo al tempo 105

Esercizio 52)

- Si realizzi mediante semafori il seguente grafo di precedenza
  - Tutti i processi devono essere **ciclici**




```
init (S1, 1);
init (S2, 0);
init (S3, 0);
init (S4, 0);
```

```
P1
while (1) {
  wait (S1);
  SC di P1
  signal (S2);
  signal (S3);
}
```

```
P2
while (1) {
  wait (S2);
  SC di P2
  signal (S4);
}
```

```
P3
while (1) {
  wait (S3);
  SC di P3
  signal (S4);
}
```

```
P4
while (1) {
  wait (S4);
  wait (S4);
  SC di P4
  signal (S1);
}
```

Soluzione errata

```
P2
while (1) {
  wait (S2);
  SC di P2
  signal (S3);
}
```

```
P3
while (1) {
  wait (S2);
  SC di P3
  signal (S3);
}
```

```
P4
while (1) {
  wait (S3);
  wait (S3);
  SC di P4
  signal (S1);
}
```

Esercizio 53)

- Dall'enunciato si sa che, nel primo disco:
  - è occupato il 10% del disco cioè circa **100** MB
  - i file di dimensione inferiore a **2** KB occupano la metà dello spazio, cioè circa **50** MB.
- Un file, anche se più piccolo di **2** KB, occupa comunque un intero cluster , cioè **2** KB
- Quindi, una stima del il numero dei file 'piccoli' è:

$$N_{small} = 50 \text{ MB} / 2 \text{ KB} = 25.000$$

- Sugli altri file non è possibile dedurre altro, se non il fatto che occupano, nella prima organizzazione, circa **50** MB

Esercizio 54)

Ricordate che si basa su FIFO

N.B. nel testo dice metodo globale, dunque anche C5 dovrebbe essere settato a 0

Svolgimento: Siccome 4 Kbyte corrispondono a 4096 byte e ogni cella è grande 2 byte, lo spazio di memoria comprenderà 2048 celle i cui indirizzi andranno da 0 a 2047.

$t = 0$ Viene eseguito P_1, e tutta la memoria risulta libera. Il processo viene dunque allocato nei primi 2048 byte, ovvero nelle celle i cui indirizzi vanno da 0 a 1023, e le celle rimanenti vanno a formare un'area di memoria libera L_1.

$t = 7$ Viene eseguito P_2, e l'unica area di memoria libera è L_1. Il processo viene quindi allocato nelle celle i cui indirizzi vanno da 1024 a 1549 (il processo è grande 1049 byte, quindi saranno necessarie 525 celle per contenerlo), mentre L_1 si riduce a occupare le celle tra gli indirizzi 1550 e 2047.

Errato, 1548 non 1549

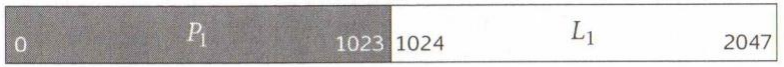
$t = 10$ Termina P_1, così che la memoria che occupava va a formare un'area libera L_2.

$t = 11$ Viene eseguito P_3, la cui dimensione di 500 byte è compatibile sia con L_1, sia con L_2 (rispettivamente di 996 e 2048 byte). L'algoritmo *best fit* selezionerà L_1, in quanto si tratta dell'area con dimensione minore: il processo verrà quindi allocato nelle celle tra gli indirizzi 1550 e 1799, mentre L_1 andrà a occupare le celle rimanenti.

$t = 13$ Viene eseguito P_4. Come per il punto precedente, entrambe le aree di memoria libera hanno dimensione sufficiente ad accogliere il processo, e quindi viene selezionata l'area più piccola: P_4 è dunque allocato dalla cella con indirizzo 1800 alla cella con indirizzo 1949, e L_1 viene ridimensionata in modo da coinvolgere le celle con indirizzo tra 1950 e 2047.

$t = 20$ Termina P_2, e l'area L_2 viene ingrandita in modo da comprendere la memoria liberata.

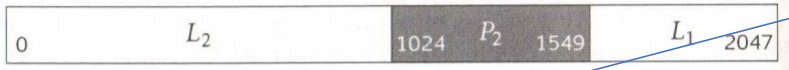
$t = 23$ Termina P_4, e la memoria precedentemente occupata viene inglobata in L_1. Analogamente a $t = 30$ termina P_3, e la memoria allocata al processo e le aree L_1 e L_2 vanno a formare un'unica zona libera.



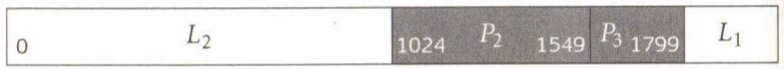
(a) $t = 0$



(b) $t = 7$



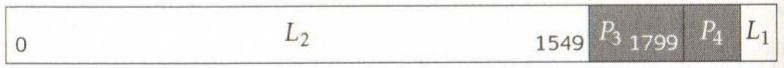
(c) $t = 10$



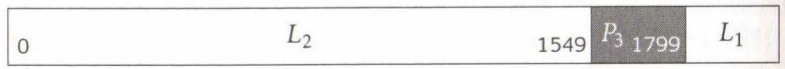
(d) $t = 11$



(e) $t = 13$



(f) $t = 20$



(g) $t = 23$

Processo	Grandezza (byte)	Tempo di esecuzione (ms)	Durata (ms)
P_1	2048	0	10
P_2	1049	7	13
P_3	500	11	19
P_4	300	13	10

Esercizio 55)

all'istante $t = 10$ la zona di memoria libera più grande è L_2. Pertanto la dimensione di tale area, che è pari a 2 Kbyte (o equivalentemente a 1024 celle), rappresenta anche la grandezza massima di un processo eseguibile.

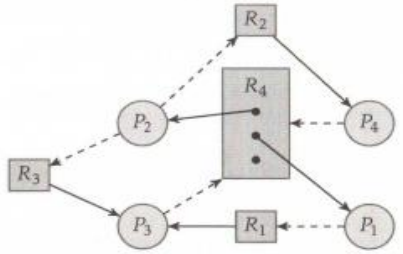
si verifica facilmente che anche all'istante $t = 13$ la dimensione massima di un processo eseguibile è di 2 Kbyte. ■

Esercizio 56)

Il grafo di allocazione contiene più cicli (come per esempio quello che coinvolge P_2, R_2, P_4 e R_4). Non essendo però tutte le risorse a istanza singola, la presenza di cicli è condizione necessaria ma non sufficiente per determinare lo stato di *deadlock*. Bisogna

quindi utilizzare un altro metodo per verificare se il sistema si trovi o meno in *deadlock*. Risulta comodo utilizzare l'algoritmo del banchiere, avendo già determinato le matrici corrispondenti. Come si può verificare dalla tabella seguente, la sequenza $< P_3, P_1, P_4, P_2 >$ è sicura. Pertanto il sistema non si trova in *deadlock*.

	Disponibili			
	R_1	R_2	R_3	R_4
	0	0	0	1
P_3	1	0	1	1
P_1	1	0	1	2
P_4	1	1	1	2
P_2	1	1	1	3



3. Un'ulteriore richiesta del processo P_3 di accedere alla risorsa R_4 è compatibile sia con la disponibilità delle risorse, sia con il numero massimo di richieste dichiarate da P_3. Va quindi determinato se il soddisfacimento di questa richiesta mantiene o meno il sistema in uno stato sicuro. Lo stato in cui si troverebbe il sistema in caso di soddisfacimento è descritto dalle seguenti matrici.

	Assegnate			
	R_1	R_2	R_3	R_4
P_1	0	0	0	1
P_2	0	0	0	1
P_3	1	0	1	0
P_4	0	1	0	0

	Massimo			
	R_1	R_2	R_3	R_4
P_1	1	0	0	1
P_2	0	1	1	1
P_3	1	0	1	1
P_4	0	1	0	1

	Necessità			
	R_1	R_2	R_3	R_4
P_1	1	0	0	0
P_2	0	1	1	0
P_3	0	0	0	1
P_4	0	0	0	1

	Disponibili			
	R_1	R_2	R_3	R_4
	0	0	0	1

Applicando l'algoritmo del banchiere si verifica come la sequenza $< P_3, P_1, P_4, P_2 >$ sia sivura, come evidenziato qui sotto.

	Disponibili			
	R_1	R_2	R_3	R_4
	0	0	0	1
P_3	1	0	1	1
P_1	1	0	1	2
P_4	1	1	1	2
P_2	1	1	1	3

Esercizio 57)

Svolgimento: Fissando convenzionalmente a $t = 0$ il tempo in cui il processo viene caricato in memoria, in tale istante lo schema di prepaginazione inserisce le pagine da 0 a 3 nei *frame* del processo. Indicando con $t = i$, per $i > 0$, l'istante in cui il processo effettua l'i-esimo accesso in memoria, l'esecuzione dell'algoritmo di rimpiazzamento è riassunta qui di seguito.

$t = 1$ La pagina 15 non si trova in memoria centrale, e tutti i *frame* contengono pagine a cui non si è mai acceduto. La pagina verrà quindi allocata nel *frame* f_0 in quanto il suo indice è quello più piccolo.

$t = 2$ La pagina a cui si accede è già caricata in memoria centrale, quindi non vi sono variazioni.

$t = 3$ La pagina 0 non è inserita in memoria centrale, e solo il *frame* f_0 contiene una pagina a cui si è precedentemente acceduto. Potendo scegliere tra i *frame* f_1, f_2 e f_3, la pagina verrà caricata in f_1 in quanto il suo indice è quello più basso.

$t = 4, 5$ Le pagine 2 e 3 si trovano già in memoria centrale, quindi non vi sono variazioni.

$t = 6$ La pagina 11 genera un *page fault*, e tra quelle caricate in memoria centrale quella acceduta meno recentemente è la pagina 15, che verrà quindi rimpiazzata dalla pagina 11.

$t = 7$ La pagina 2 è già presente in memoria.

$t = 8, 9, 10$ Le pagine 6, 4 e 14 non sono presenti in memoria centrale, e l'algoritmo LRU le sostituisce rispettivamente alle pagine 0, 3 e 11.

Vengono dunque causati sei *page fault*.

t	0	1	2	3	4	5	6	7	8	9	10
Pagina	15	15	0	2	3	11	2	6	4	14	
f_0	0	15	15	15	15	15	11	11	11	11	14
f_1	1	1	1	0	0	0	0	0	6	6	6
f_2	2	2	2	2	2	2	2	2	2	2	2
f_3	3	3	3	3	3	3	3	3	3	4	4

15, 15, 0, 2, 3, 11, 2, 6, 4, 14.

Esercizio 58)

Svolgimento: Prendendo in considerazione le pagine elencate ed applicando l'algoritmo di rimpiazzamento ottimale è possibile rispondere alle domande poste nel modo seguente.

1. Indicando con $t = i$, per $i > 0$, l'istante in cui il processo effettua l'i-esimo accesso in memoria, l'esecuzione dell'algoritmo di rimpiazzamento è riassunta qui di seguito.

   $t = 1$ La pagina 6 non si trova in memoria centrale, e tutti i *frame* sono liberi: la pagina verrà quindi allocata nel *frame* f_0 in quanto il suo indice è quello più piccolo.

   $t = 2$ La pagina a cui si accede è già caricata in memoria centrale, quindi non vi sono variazioni.

   $t = 3$ La pagina 2 non è inserita in memoria centrale: essendo liberi i *frame* f_1 e f_2, la pagina verrà caricata nel primo di questi.

   $t = 4$ La pagina 12 non è presente in memoria, e solo il *frame* f_2 è libero, quindi è in questo *frame* che verrà caricata la pagina.

   $t = 5$ La pagina 7 genera un *page fault*, ma in questo caso non vi sono *frame* liberi. In memoria sono presenti le pagine 6, 2 e 12, a cui si accederà rispettivamente agli istanti 10, 6 e 8. La pagina a cui si accederà più in là nel tempo è quindi quella contenuta nel *frame* f_0, ed è in questo *frame* che verrà inserita la pagina cui si accede.

   $t = 6$ La pagina cui si accede è già in memoria, quindi non vengono generati *page fault*.

   $t = 7$ L'accesso alla pagina 0 genera un *page fault*. Le pagine 2 e 12 verranno accedute rispettivamente negli istanti 9 e 8, mentre non si accederà più alla pagina 7. Quindi il rimpiazzamento coinvlgerà il *frame* f_0.

   $t = 8, 9$ L'accesso alle pagine 12 e 2 non genera *page fault*.

   $t = 10$ L'accesso alla pagina 6 genera un *page fault*. Essendo tale accesso l'ultimo fatto dal processo, non si accederà in futuro ad alcuna delle pagine in memoria. La politica di rimpiazzamento ottimale può quindi scegliere di inserire la pagina 6 in un *frame* qualsiasi.

   Vengono dunque causati sei *page fault*.

t	1	2	3	4	5	6	7	8	9	10
Pagina	6	6	2	12	7	2	0	12	2	6
f_0	6	6	6	6	7	7	0	0	0	6
f_1	–	–	2	2	2	2	2	2	2	2
f_2	–	–	–	12	12	12	12	12	12	12

2. Essendo la paginazione a richiesta pura, il numero minimo di *page fault* deve essere pari al numero di pagine diverse a cui il processo accede. Siccome tale numero è pari a sei, anche aumentando il numero di *frame* non potrà diminuire il numero di *page fault*.

3. La politica di rimpiazzamento ottimale non soffre dell'anomalia di Belady, quindi per una generica configurazione di accessi alle pagine, aumentando il numero di *frame* non può diminuire il numero di *page fault*. Va sottolineato che questa argomentazione risponde anche alla domanda precedente.

In realtà dal testo accede a 5 pagine diverse, dunque il numero minimo di page fault è 5, non 6

"aumentando il numero di frame non può *aumentare* il numero di page fault"

Esercizio 59)

1. Siccome una cella di memoria occupa 32 bit, che corrispondono a 4 byte, e la grandezza delle pagine e dei *frame* è di un Kbyte che corrisponde a 1024 byte, all'interno di pagine e *frame* ci saranno $1024/4 = 256 = 2^8$ diverse celle. Pertanto saranno necessari 8 bit nell'indirizzo per poter specificare tutti i valori possibili per lo scostamento.

2. Dal punto precedente si ottiene che dei 32 bit che compongono l'indirizzo di una cella, 8 vanno dedicati allo scostamento. Quindi i 24 bit rimanenti possono essere utilizzati per specificare il numero di pagina. Pertanto in linea di principio è possibile avere dei processi con 2^{24} pagine differenti, per un totale di $2^{24} \cdot 1024 = 2^{34} = 16 \cdot 2^{30}$ byte, cioè 16 Gbyte.

3. Considerando gli indirizzi logici indicati, si verificano i seguenti casi:

   (a) Dividendo l'indirizzo logico $(0\ldots0\ 0111\ 1101\ 0101)_2$ nelle parti che comprendono i 24 bit più significativi e gli otto meno significativi, eliminando gli zero iniziali si ottiene $(111)_2 = 7$ in

corrispondenza del numero di pagina e $(1101\ 0101)_2 = 213$ per quanto riguarda lo scostamento. Dalla tabella delle pagine si verifica che la pagina 7 è allocata nel *frame* 106, per cui l'indirizzo fisico a cui si accederà sarà

```
106 * 256 + 213 = 27349 cella
```

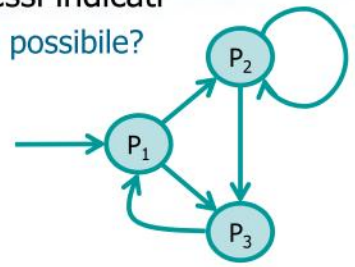
(b) L'indirizzo logico $(0\ldots0\ 0100\ 0001\ 0111)_2$ si riferisce alla pagina $(100)_2 = 4$, ma in corrispondenza di quest'ultima si verifica che la tabella delle pagine ha il bit di validità pari a 0. Pertanto l'accesso a questo indirizzo logico genera un'assenza di pagina che andrà gestita dal *pager*.

(c) Analizzando l'indirizzo logico $(0\ldots0\ 1011\ 1101\ 1011)_2$ si verifica come la pagina a cui questo fa riferimento sia $(1011)_2 = 11$, il cui numero non compare nella tabella delle pagine: l'accesso è quindi invalido e il gestore della memoria genererà una *trap*.

N. di pagina	bit di validità	N. di *frame*
0	1	739
1	1	127
2	0	–
3	1	91
4	0	–
5	1	470
6	1	2
7	1	106
8	1	495
9	0	–

Esercizio 60)

- Siano dati i semafori e i processi indicati
  - Quale ordine di esecuzione è possibile?




```
init (S1, 1);
init (S2, 0);
```

P1
```
...
while (1) {
  wait (S1);
  SC di P1
  signal (S2);
}
...
```

P2
```
...
while (1) {
  wait (S2);
  SC di P2
  signal (S2);
}
...
```

P3
```
...
while (1) {
  wait (S2);
  SC di P3
  signal (S1);
}
...
```

Esercizio 61)

T (slice 200)	CPU	Queue
0	P1<500>	
100	P1<400>	P2<300>
200	P2<300>	P1<300>
300	P2<200>	P1<300>, P3<400>
400	P1<300>	P3<400>, P2<100>
600	P3<400>	P2<100>, P4<1000>, P1<100>
800	P2<100>	P4<1000>, P1<100>, P5<600>, P3<200>
900 – P2 ends	P4<1000>	P1<100>, P5<600>, P3<200>
1100	P1<100>	P5<600>, P3<200>, P4<800>
1200 – P1 ends	P5<600>	P3<200>, P4<800>
1400	P3<200>	P4<800>, P5<400>
1600 – P3 ends	P4<800>	P5<400>
1800	P5<400>	P4<600>
2000	P4<600>	P5<200>
2200	P5<200>	P4<400>
2400 – P5 ends	P4<400>	
2800 – P4 ends		

Process	Tarrive	Tend	$T_{turnaround}$	Trun	Twait	Tturn(norm)
P1	0	1200	1200	500	700	1200/500 = 2,4
P2	100	900	800	300	500	800/300 = 2,67
P3	300	1600	1300	400	900	1300/400 = 3,25
P4	600	2800	2200	1000	1200	2200/ 1000 = 2,2
P5	700	2400	1700	600	1100	1700/600 = 2,83
Average					880	2,67

T (slice 500)	CPU	Queue
0	P1<500>	
100	P1<400>	P2<300>
300	P1<300>	P2<300>, P3<400>
500 – P1 ends	P2<300>	P3<400>
600	P2<200>	P3<400>, P4<1000>
700	P2<100>	P3<400>, P4<1000>, P5<600>
800 – P2 ends	P3<400>	P4<1000>, P5<600>
1200 – P3 ends	P4<1000>	P5<600>
1700	P5<600>	P4<500>
2200	P4<500>	P5<100>
2700 – P4 ends	P5<100>	
2800 – P5 ends		

Process	Tarrive	Tend	$T_{turnaround}$	Trun	Twait	Tturn(norm)
P1	0	500	500	500	0	500/500 = 1
P2	100	800	700	300	400	700/300 = 2,33
P3	300	1200	900	400	500	900/400 = 2,25
P4	600	2700	2100	1000	1100	2100/1000 = 2,1
P5	700	2800	2100	600	1500	2100/600 = 3,5
Average					700	2,23

Esercizio 62)

a) Looking at the content of RAM and considering that the page table of the process has a descriptor per page, the answer is:

Page	Frame	Valid Bit
0		0
1	3	1
2	-	0
3	-	0
4	0	1
5	2	1
6	-	0
7	-	0
8	-	0
9	1	1
10	-	0
11	-	0
12	-	0
13	-	0
14	-	0
15	-	0

b) Address translation is made looking up the page table:

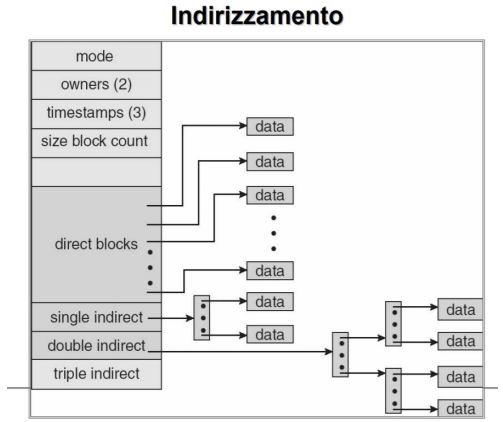
Page = logical address / page size = logical address / 512
Offset = logical address *mod* page size
Physical address = Frame * 512 + offset

Logical address	Page	Offset	Physical address
1628	3		
851	1	339	1875
2700	5	140	1164
2432	4	384	384

c) If the logical address 1330 is referenced, a page fault is obtained because this logical address belong to page 2, whose validity bit in 0 in the page table. Thus, page 2 is not in RAM memory and the memory manager raises a page fault.

la pagina 3 non è in memoria. l'accesso a quell'indirizzo dovrebbe scatenare un page fault.

Esercizio 63)



In questo caso il file occupa 50 blocchi, nel testo dell'esercizio dice che ho 10 puntatori diretti, poi devo usare livelli di indirizzamento (vedi dispense). Ogni blocco indicizzato dai livelli di indirizzamento contiene 1024 address

In questo caso devo considerare un secondo livello, il primo permette solo fino a 1024 indirizzi

In each address block 1024 block addresses can be allocated: 4KBytes /4 bytes =1024 addresses

Il blocco è 4 KB e ogni address è 4 bytes, dunque divido per sapere quanti Indirizzi possono essere allocati

a. Number of blocks

- File A with a size of 20 KBytes

A 20KB file will use 5 data blocks, using the first 5 direct pointers.

That is, it will use **5 blocks**

Data la dimensione di un file, basta dividere per 4KB (dimensione di un blocco) per sapere quanti blocchi occupa

- File B with a size of 200 KBytes

A 200Kb file will use 50 data blocks. The first 10 will be addressed by the direct pointers. The remaining 40 will be pointed by the single indirect pointers (an extra block is needed).

That is, it will use 50 blocks + 1 block (related to the single indirect pointer) = **51 blocks**

- File C with a size of 2000 KBytes

A 2000KB file will use 500 data blocks. The first 10 will be addressed by the direct pointers. The remaining 490 will be pointed by the single indirect pointers (an extra block is needed).

That is, it will use 500 blocks + 1 block (related to the single indirect pointer) = **501 blocks**

- File D with a size of 20 000 Kbytes

A 20000KB file will use 5000 data blocks. The first 10 will be addressed by the direct pointers. The next 1024 blocks will be pointed by the single indirect pointers (an extra block is needed). The remaining 3966 blocks will be addressed using the double indirect pointers (using 1 block to keep the pointer block addresses (1024 address block) plus 4 blocks to point the 3966 data blocks.

That is: 5000 blocks + 1 block (single indirect) + 5 blocks (double indirect) = **5006 blocks**

c) What is the maximum possible file size?

The maximum number of blocks of a file is:

- 10 blocks addressed by direct pointers
- 1024 blocks addressed by single indirect pointers
- 1024*1024 blocks addressed by double indirect pointers
- 1024*1024*1024 blocks addressed by triple indirect pointers

The overall number of block is 1G+1M+1K+10 blocks which is approximately **4 TBytes** (not considering 4GB, 4MB and 40KB because are much smaller than the first term).

Esercizio 64)

È sufficiente che: Burst_P1 ≤ Burst_P2 ≤ Burst_P3

Esercizio 65)

33. Ogni blocco infatti memorizza 1020 byte di dati più 4 byte di puntatore al blocco successivo (infatti, $2^{40}/2^{10} = 2^{30}$), per cui sono necessari 33 blocchi per memorizzare l'intero file (possiamo alternativamente osservare che se potessimo usare un intero blocco per i dati del file, ci vorrebbero $2^{15}/2^{10} = 2^5 = 32$ blocchi per memorizzare il file, ma poiché in ognuno dei 32 blocchi 4 byte sono usati per contenere il puntatore al blocco successivo, i blocchi effettivi in cui viene memorizzato il file sono 33)

Esercizio 66)

Poiché 8 Kbyte = 2^{13} byte, il numero di un frame del sistema è scritto su 29 – 13 = 16 bit, ossia 2 byte, dunque in un singolo frame possiamo scrivere al massimo 4096 = 2^{12} entry da due byte, e lo spazio di indirizzamento logico è pari a $2^{12} * 2^{13} = 2^{25}$ byte.

Esercizio 67)

Si osservi innanzi tutto che lo spazio di indirizzamento logico è pari a 2^{33} byte.

Una page table può avere al massimo $2^{33} / 2^{20}$ entry, mentre il numero massimo di frame del sistema è pari a $2^{32} / 2^{20} = 2^{12}$. Quindi, abbiamo bisogno di 12 bit per scrivere il numero di un frame. Se per semplicità assumiamo che si usino due byte per ciascuna entry di una page table, la sua dimensione massima sarà di di $2* 2^{33} / 2^{20} = 2^{14}$ byte = 16 Kbyte

Esercizio 68)

```
#define CHAIRS 5                    /* # chairs for waiting customers */

typedef int semaphore;              /* use your imagination */

semaphore customers = 0;            /* # of customers waiting for service */
semaphore barbers = 0;              /* # of barbers waiting for customers */
semaphore mutex = 1;                /* for mutual exclusion */
int waiting = 0;                    /* customers are waiting (not being cut) */

void barber(void)
{
     while (TRUE) {
          wait  (&customers);       /* go to sleep if # of customers is 0 */
          wait  (&mutex);           /* acquire access to 'waiting' */
          waiting = waiting – 1;    /* decrement count of waiting customers */
       signal (&barbers);           /* one barber is now ready to cut hair */
       signal (&mutex);             /* release 'waiting' */
          cut_hair( );              /* cut hair (outside critical region) */
     }
}

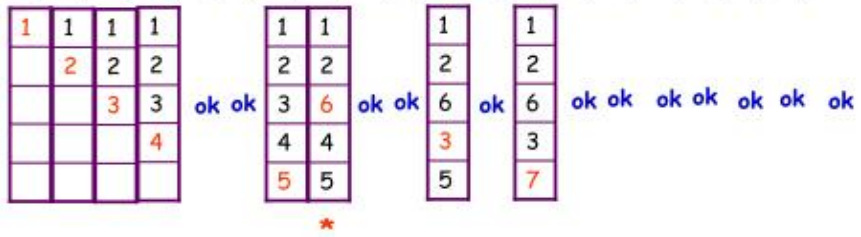

void customer(void)
{
      wait  (&mutex);               /* enter critical region */
     if (waiting < CHAIRS) {        /* if there are no free chairs, leave */
          waiting = waiting + 1;    /* increment count of waiting customers */
       signal (&customers);         /* wake up barber if necessary */
       signal (&mutex);             /* release access to 'waiting' */
          wait  (&barbers);         /* go to sleep if # of free barbers is 0 */
        get_haircut( );             /* be seated and be serviced */
     } else {
       signal (&mutex);             /* shop is full; do not wait */
     }
```

Esercizio 69)

pagine da 2^{10} byte: una page table può avere fino a $2^{33} / 2^{10} = 2^{23}$ entry. Lo spazio di indirizzamento fisico è suddiviso in $2^{32} / 2^{10} = 2^{22}$ frame, e 3 byte sono sufficienti per scrivere il numero di un qualsiasi frame. Al massimo una page table occupa quindi $3 * 2^{23}$ byte = pari a circa 24 Mbyte. Poiché un frame è molto più piccolo, la paginazione su più livelli potrebbe essere necessaria.

Esercizio 70)

1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 3, 2, 7, 6, 3, 2, 1, 2, 3, 6

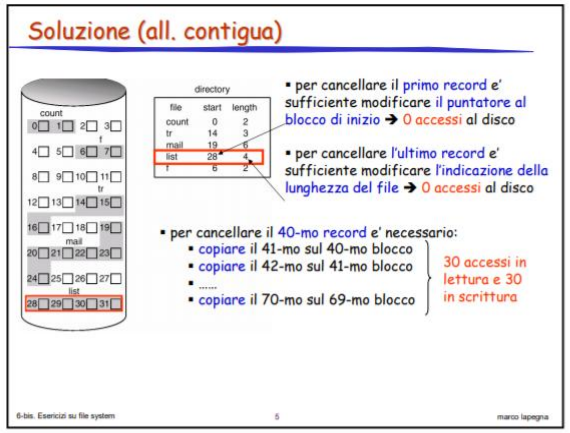


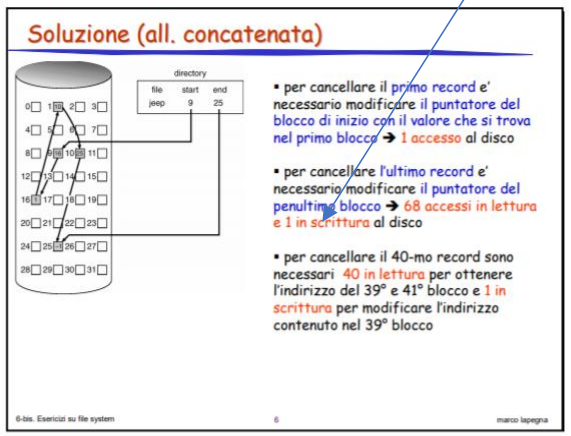
* Quando la pagina 6 prende il posto della pagina 3, quest'ultima deve essere salvata perche' l'accesso precedente era in scrittura

- 1 page fault con salvataggio della pagina sostituita
- 7 page fault senza salvataggio della pagina sostituita
- 12 accessi diretti senza page fault

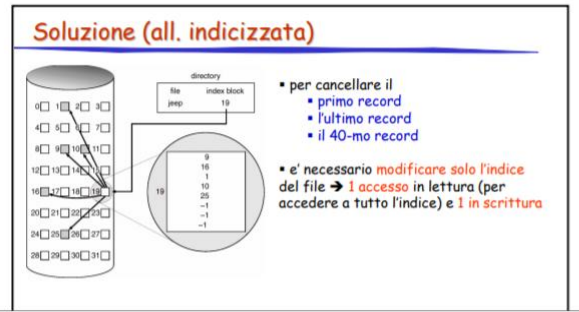
- EAT = (12/20 * 80 + 7/20 * 80000 + 1/20 * 140000) *10^{-6} sec = 35.048 msec

Esercizio 71)





1 Lettura e 1 scrittura, non solo 1 scrittura



Esercizio 72)

Risulta:
Testo 200KB → 200/16 = 12.5 = 13 pagine
Dati 100KB → 100/16 = 6.25 = 7 pagine
Stack 50KB → 50/16 = 3.125 = 4 pagine
Per un totale di **24 pagine**

Per indirizzare pagine da 16KB (16K=2^{14}), c'è bisogno di 14 bit.
Restano 18 bit per generare i numeri delle pagine → 256K pagine

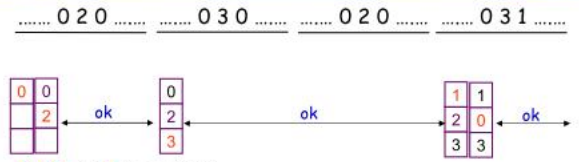
Quindi 2^{32} = 4GB di memoria complessiva.

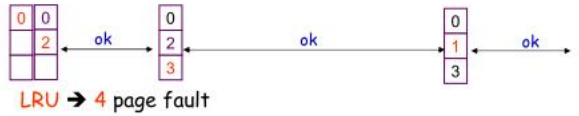
Esercizio 73)

- Gli array A e C occupano 1024B ciascuno, suddivisi in 2 pagine da 512B come segue:
  - A[0..255] alla pagina 0, A[256..511] alla pagina 1
  - C[0..255] alla pagina 2, C[256..511] alla pagina 3
- Riferimenti generati dalla CPU per eseguire A[i*j] = A[2*i] + C[2*j];
  - i=1
    - j=0 0 2 0
    - ...
    - j=127 0 2 0
    - j=128 0 3 0
    - ...
    - j=255 0 3 0
  - i=2
    - j=0 0 2 0
    - ...
    - j=127 0 2 0
    - j=128 0 3 1
    - ...
    - j=255 0 3 1

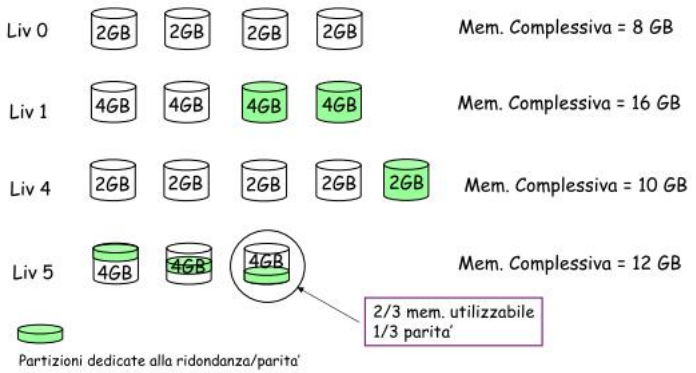
..... 0 2 0 .... 0 3 0 .... 0 2 0 .... 0 3 1 .....

128 volte 128 volte 128 volte 128 volte





Esercizio 74)



Esercizio 75)

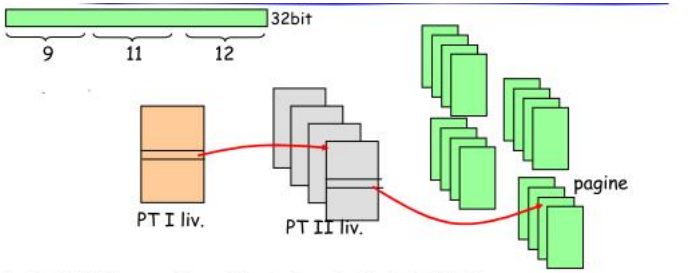
```
semaphore bus = 30;
semaphore door = 2;


getting_on(){

P(&bus);
P(&door);
walk_in();
V(&door);

}//getting on
getting_off(){
P(&door);
walk_out();
V(&door);
V(&bus)
```

Esercizio 76)



La PT di I liv. contiene 2^9 entries ➔ 2^9 PT di II liv.
Ogni PT di II liv. contiene 2^{11} entries ➔ 2^{11} pagine per ogni PT di II liv.

Totale di $2^9 \times 2^{11} = 2^{20}$ pagine ~ 10^6 pagine

Dimensione di ciascuna pagina: $2^{12} = 4 \times 2^{10} = 4K$

Esercizio 77)

Le pagine sono da 512B e ogni intero è 4B (128 elementi a pagina):

0	1	2	3
a[0] ... a[127]	a[128] ... a[255]	a[256] ... a[383]	a[384] ... a[511]

for (i=0; i < N/2; i++) a[i] = a[2*i] + a[N-i-1];

i = 0,... 63	a[2*i] -> 0 a[N-i-1] -> 3 a[i] -> 0	64 volte
i = 64,...127	a[2*i] -> 1 a[N-i-1] -> 3 a[i] -> 0	64 volte
i = 128,...191	a[2*i] -> 2 a[N-i-1] -> 2 a[i] -> 1	64 volte
i = 192,...255	a[2*i] -> 3 a[N-i-1] -> 2 a[i] -> 1	64 volte

Totale riferimenti = 768

64*4*3=768

0	3	0	...	...	1	3	0	...	...	2	2	1	...	...	3	2	1	...	...
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	3	3	3	3	3
	3	3	3	3	3	3	3	3	3	3	3	1	1	1	1	1	1	1	1
					1	1	1	1	1	2	2	2	2	2	2	2	2	2	2
					ok	ok	ok		ok	ok	ok	ok		ok		ok	ok		ok	ok	ok	ok

- 6 page fault
- 762 riferimenti validi

EAT = (6/768 * 25*10^{-3} + 762/768 *100*10^{-6}) sec = 0.294*10^{-3}

Effective Access Time (EAT)

Esercizio 78)

- (a) $2^6 = 64$.
- (b) $2^4 = 16$.
- (c)
  - $9 \rightarrow (0, 9) \rightarrow 200 + 9 = 209$;
  - $132 \rightarrow (2, 4) \rightarrow 252 + 4 = 256$;
  - $79 \rightarrow (1, 15) \rightarrow 100 + 15 = 115$;
  - $64 \rightarrow (1, 0) \rightarrow 100 + 0 = 100$;
  - $259 \rightarrow (4, 3) \rightarrow 683 + 3 = 686$;
  - $135 \rightarrow (2, 7) \rightarrow$ segmentation fault;
  - $320 \rightarrow (5, 0) \rightarrow$ invalid segment;

Esercizio 79)

```
vigile() {
     while(1) {
        sleep(T);
        wait(NS);
        wait(incrocio);
        signal(incrocio);
        signal(EW);

        sleep(T);
        wait(EW);
        wait(incrocio);
        signal(incrocio);
        signal(NS);
     }
}
```

Note:

a) L'inizializzazione assicura che all'inizio abbiano il via le macchine sulla direttrice NS

b) mutexNS e mutexEW fanno sì che una sola macchina per volta si accodi sui semafori NS ed EW, in modo che il vigile possa bloccare al più presto il semaforo allo scadere del tempo, senza dover aspettare molte macchine già in coda (importante curare questo aspetto).

c) Il semaforo incrocio consente al vigile di aspettare che l'incrocio sia libero prima di dare il via libera sull'altra direzione (questo requisito è importante).

Esercizio 80)

Soluzione variando il numero di frame.
Nel testo chiedeva solo per 5 frames. Inoltre presenti anche soluzioni con algoritmo ottimo.

Number of frames	LRU	FIFO	Optimal
1	20	20	20
2	18	18	15
3	15	16	11
4	10	14	8
5	8	10	7
6	7	10	7
7	7	7	7

Esercizio 81)

a. Logical address: 16 bits

b. Physical address: 15 bits

1024=2^10 parole,
Dunque 10 bit per offset.
64=2^6 pagine, dunque 6 bit per indicizzarle.

Nel caso della memoria fisica abbiamo 2^5 frames -> 5 bit per indicizzarli.

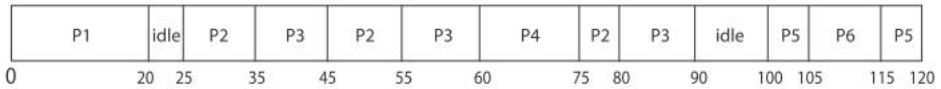
Esercizio 82)

Conventional, single-level page table will have $2^{10} = 1024$ entries. Inverted page table will have $2^5 = 32$ entries. The maximum amount of physical memory is $2^{16} = 65536$ (or 64 KB.)

Pagine da 2048=2^11 byte; 21 bit di indirizzo virtuale, di questi 11 servono per offset, dunque 10 per indicizzare le pagine (2^10 pagine indicizzabili).
Indirizzi fisici sono 16 bit, sempre 11 per offset, dunque avrò 2^5 pagine fisiche indicizzabili.

Esercizio 83)

a. The Gantt chart:



b. P1: 20-0= 20, P2: 80-25 = 55, P3: 90 - 30 = 60, P4: 75-60 = 15, P5: 120-100 = 20, P6: 115-105 = 10

c. P1: 0, p2: 30, P3: 35, P4: 0, P5: 10, P6: 0

Esercizio 84)

a. n

b. p

n perché devo caricarli tutti in memoria
p nel caso peggiore ogni riferimento provoca un page fault

Esercizio 85)

```
type A = array[1..10] of orange;
var pieno: semaphore(initial 1);
    vuoto: semaphore(initial 0);
    mutex: semaphore(initial 1);
    index: 1..10;
    aranciate: A;

Procedure carica(x:A);
begin
   wait(pieno);
   aranciate := x;
   index := 10;
   signal(vuoto)
end;
```

```
Procedure prelevai(var y: orange);
begin
   wait(mutex);
   wait(vuoto);
   y := aranciate[index];
   index := index - 1;
   if index = 0
      then signal(pieno)
      else signal(vuoto);
   signal(mutex)
end;
```

Esercizio 86)

siccome $8 = 2^3$, $1024 = 2^{10}$ e $32 = 2^5$, si ha:

(a) indirizzo logico: 3 (n. di pagina) + 10 (offset nella pagina) = 13 bit,

(b) indirizzo fisico: 5 (n. del frame) + 10 (offset nel frame) = 15 bit.

Esercizio 87)

$WS(10, 8) = \{1, 4, 5, 6, 7\}$

8 page fault. Basta fare la matrice seguente, facendo attenzione a togliere le pagine man mano che escono dal working set, e segnando fault quando bisogna (ri)mettercele:

2	6	5	7	7	7	7	5	1	6	4
2	6	5	7	7	7	7	5	1	6	4
	2	6	5	5	5		7	5	1	6
		2	6	6				7	5	1
			2						7	5
p	p	p	p				p	p	p	p

Esercizio 88)

$\frac{\frac{x}{y}}{z} = \frac{x \times z}{y}$

infatti basta notare che $x = \frac{x}{1}$, quindi

$\frac{\frac{x}{y}}{z} = \frac{\frac{x}{1}}{\frac{y}{z}} = \frac{x \times z}{y}$

Il tempo medio necessario per soddisfare la prima richiesta sarà dato dalla somma del tempo medio di seek, del tempo di latenza medio (in generale metà del tempo necessario a compiere una rotazione completa del disco) e del tempo di trasferimento. Quindi $t_{\text{medio}} = t_{\text{seek}} + t_{\text{latenza}} + t_{\text{trasferimento}}$, dove $t_{\text{seek}} = 3$ ms, $t_{\text{latenza}} = \frac{60 \cdot 10^3}{2 \cdot 7200} \equiv 4,17$ ms e $t_{\text{trasferimento}} = \frac{4 \cdot 1024 \cdot 10^3}{40 \cdot 1024^2} = 0,098$ ms. Quindi $t_{\text{medio}} = 7,268$ ms.

Per quanto riguarda la seconda richiesta invece, dato che la testina si trova ancora sulla stessa traccia, non bisogna tenere conto del tempo medio di seek. Quindi $t'_{\text{medio}} = t'_{\text{latenza}} + t'_{\text{trasferimento}}$, dove $t'_{\text{latenza}} = \frac{60 \cdot 10^3}{2 \cdot 7200} \equiv 4,17$ ms e $t'_{\text{trasferimento}} = \frac{4 \cdot 1024 \cdot 10^3}{40 \cdot 1024^2} = 0,098$ ms. Da cui si ricava $t'_{\text{medio}} = 4,268$ ms.

Quel "cubo" al numeratore è per passare a millisecondi. Nel denominatore 1024 al quadrato è un megabyte

Esercizio 89)

(a) La posizione 10000 cade nel terzo blocco, che è uno dei blocchi diretti. Per cui basta 1 accesso al disco (l'inode è già stato caricato in memoria al momento dell'apertura), che costa $t_{seek} + t_{latenza} = 10 + 60/(2 * 7, 2) = 14, 17msec.$

(b) La posizione 100000 cade in uno dei primi indiretti, per cui è necessario accedere 2 volte al disco (una volta anche per il blocco indiretto), dove però il tempo di seek si conta una volta sola perché i blocchi sono sulla stessa traccia. In totale $10 + 4, 17 + 4, 17 = 18, 34msec.$

Esercizio 90)

- 0, 430 corrisponde all'indirizzo fisico 219+430=649 (indirizzo valido dato che 430<600);
- 1, 10 corrisponde all'indirizzo fisico 2300+10=2310 (indirizzo valido dato che 10<14);
- 2, 500 non corrisponde ad un indirizzo fisico valido dato che 500>100;
- 3, 400 corrisponde all'indirizzo fisico 1327+400=1727 (indirizzo valido dato che 400<580);
- 4, 112 non corrisponde ad un indirizzo valido dato che 112>96.

Esercizio 91)

```
1  print() {
2    sem go2 = 0, go3 = 0;
3
4    process P1 {
5      write (''line 1''); write (''line 2'');
6      V(go2);
7    }
8
9    process P2 {
10     P(go2);
11     write (''line 3''); write (''line 4'');
12     V(go3);
13   }
14
15   process P3 {
16     P(go3);
17     write (''line 5''); write (''line 6'');
18   }
19 }
```

Esercizio 92)

1. Deadlock: The processes execute the first P operation. Then both will be stuck trying to execute the second. It's the classical situation (as in the symmetric philosphers) where the lock-taking of two processes (or more) goes in different orders. If one has to look for deadlock, that's where one has to look for, P-operations of different order. The V-operations are irrelevant for deadlocks.

   Of course, the processes may also *not* deadlock.

2. The state at the deadlocked point is $(x, y, z) = (2, 1, 2)$. Up-to that point (if the program reaches the deadlock), there had been no races and therefore the result is unique.

Esercizio 93)

a. 2^{20} entries.

b. 512 K K/4K = 128K entries.

Esercizio 94)



s = sd ×(1-h) + sc × h

sc = (6-15×0.2)/0.8 = 3.75
h = (15-4.2)/(15-3.75) = 0.96

s = tempo di servizio
sd = tempo di servizio disco
sc = tempo di servizio cache
h = probabilità di hit

Esercizio 95)

(a) Le coppie di valori (i, t_i) in ogni riga della tabella seguente indicano l'istante di tempo (t_i) con cui la i-esima richiesta viene soddisfatta:

Numero richiesta	Tempo di completamento
1	22
4	27
3	32
5	72
7	102
8	112
2	127
6	132

Esercizio 96)

Il codice ha una race-condition potenziale nei due statement:

```
t = conta;
conta = t + 1;
```

che porta a risultati diversi a seconda di come i processi si alternano nell'esecuzione. Se tutti i processi riescono ognuno ad eseguire i due statement senza interruzioni, la variabile conta subirà gli incrementi effettuati da tutti i processi avviati. Se invece un processo viene interrotto subito dopo il primo dei due statement, quando riprenderà ed eseguirà il secondo vanificherà l'effetto di tutti gli incrementi operati contemporaneamente dagli altri. Il risultato è imprevedibile a priori, ma il minimo corrisponderà ad una situazione di interruzione ad ogni iterazione, e sarà ovviamente pari a 20; il massimo corrisponderà alla situazione nella quale tutti gli incrementi operati vanno a buon fine, e sarà pari a 20*M.

Esercizio 97)

- 16 indici diretti di blocco, al costo di 1 blocco poiché un *i-node* occupa lo stesso spazio di un blocco;
- 1 indice di I indirezione, il quale punta ad un *i-node* interamente utilizzato per contenere indici diretti di blocco, che consente di esprimere fino a: $\lfloor \frac{1\,kB}{4\,B} \rfloor = \lfloor \frac{2^{10}}{2^2} \rfloor = 2^8 = 256$ indici di blocco, al costo di 1 ulteriore blocco;
- 1 indice di II indirezione, il quale punta ad un *i-node* speciale, interamente utilizzato per contenere puntatori ad *i-node* di I livello, che dunque consente di esprimere 256 puntatori a strutture ciascuna contenente fino a 256 indici diretti di blocco, per un totale di $256^2 = (2^8)^2 = 2^{16} = 65.536$ blocchi

dimensione massima pari a: $(16 + 256 + 65.536) \times 1\,kB =$

Esercizio 98)

16 GB = 2^{34} B diviso in blocchi da 1 KB = 2^{10} B/blocco
ovvero 2^{34} B / 2^{10} B/blocco = 2^{24} blocchi = 16 M blocchi.

Ogni blocco può contenere 1 KB / 4 B/indice = 256 indici di blocco di cui 1 viene usato come collegamento al "blocco di indici" successivo nella lista. Ne rimangono dunque 255 utilizzabili per rappresentare i blocchi liberi.

Per rappresentare una lista di **massima ampiezza** servono dunque:
16 M indici / 255 indici/blocco = 65793,0039.. ≈ **65794** blocchi
cioè poco più di 64 K × 1 KB = **64 MB**

Con la struttura a *bitmap* sono invece **sempre** necessari 2^{24} *bit* = 2^{21} B = = 2^{11} KB = **2 MB**

Esercizio 99)

Problema

Inizializzaz.
```
x = 1;
y = 2;
z = 1;
Sem1 = 0;
Sem2 = 0;
Sem3 = 0;
```

```
Process A {
P(Sem1);
z = (x - z)*y ; =3
x = x+z+y; =9
V(Sem3);
P(Sem1);
x = x + y; ?? =12
Print(x); «12»
}
```

```
Process B {
P(sem2);
x = x + y; =3
z = x - z; =2
y=(y-z)+x; =3
Print(y); «3»
V(sem1);
}
```

```
Process C {
V(Sem2);
P(Sem3);
x = x / y ; =3
y = 2z + x ; =9
V(Sem1);
z = x + z; ??
Print(z); «??»
}
```

- Il processo B stampa la variabile y per primo.
- Non possiamo fare affermazioni su quale processo stampa per secondo e per terzo tra A e C.
- Il risultato finale sarà x = 12 e y = 3 ma non possiamo dire se sarà z = 6 oppure z = 15 (a seconda di quale processo avanza per primo tra A e C)

Esercizio 100)

dimensione pagina=2^{10} byte; numero di frame = 2^8 => indirizzo fisico di 18 bit

Esercizio 101)

Process A {	Process B {	Process C {
P(semB); V(semA);	V(semB);	P(semA);
P(mutex); y = y + 2*x V(mutex);	P(mutex); x = x + 1 V(mutex);	P(mutex); x = y*3; V(mutex);
V(semA);	P(semA); V(semC);	P(semC);
	Print(x);	Print(y);
}	}	}

Esercizio 102)

B: 18 KB

Esercizio 103)

Numerosi operai in una fabbrica preparano un unico prodotto con l'utilizzo di **nA** quantità del componente **A**, **nB** quantità del componente **B**. Un fattorino viene chiamato a riempire le quantità di **A** e **B** fino a **totA** e **totB** ogni volta che le loro quantità residue scendono sotto **nA** ed **nB**. Prima di iniziare a comporre il prodotto, ogni operaio si assicura di avere le quantità necessarie dei due componenti; viceversa chiama il fattorino e attende che arrivi con la scorta. Una volta terminato il suo compito il fattorino resta in attesa di essere richiamato. Una volta completato un prodotto ogni operaio inizia a prepararne un altro. Inizialmente le scorte di **A** e **B** sono piene e nessun prodotto è stato ancora preparato.

Scrivere le seguenti procedure che propongono una soluzione a questo problema utilizzando i costrutti dei monitor.

Il monitor potrebbe utilizzare i seguenti dati:

operaio: variabile condition sulla quale gli operai si sospendono in attesa di essere riforniti di un determinato componente;
fattorino: variabile condition sulla quale il fattorino si sospende in attesa di essere mandato a comprare le scorte di un componente;
operai_in_attesa: intero che indica il numero degli operai in attesa di un componente;
nA, **nB**: constanti che indicano la quantità necessaria di ogni componente per la preparazione di un prodotto;
totA, **totB**: costanti che indicano il numero massimo di scorte per ogni componente;
qA, **qB**: interi che indicano la quantità di componenti **A** e **B** attualmente presenti in fabbrica.

```
monitor Fabbrica {
condition operaio, fattorino;
int qA, qB;
int operai_in_attesa;
```

Funzione invocata da ciascun operaio all'infinito

```
void prepara_prodotto {
   while (true) {
      if (qA > nA) and (qB > nB) {
         qA = qA - nA;
         qB = qB - nB;
      else {
         fattorino.signal();
         operai_in_attesa++;
         operaio.wait();
      }
   }
}
```

Funzione invocata da fattorino

```
void scorta_ingrediente {
     qA = totA;
     qB = totB;
     while (operai_in_attesa > 0 ) {
          operaio.signal();
          operai_in_attesa --;
     }
     fattorino.wait();
}
```

Ricordate: "Se il processo P invoca signal() su una variabile condition ed esiste un processo Q in coda su quella variabile, ci sono due opzioni:
a) Segnalare ed attendere: nel monitor entra Q e P attende (fuori dal monitor) che Q abbia terminato o si metta in attesa su una condition
b) Segnalare e proseguire: Q attende che P lasci il monitor o si metta in attesa su una condition "
In questo caso applichiamo b)

Ricordare che le due procedure implementate sono quelle relative al monitor, poi ci sarà processo esterno che le richiama. In questo caso basta richiamare prepara_prodotto, poi ci penserà questa a chiamare il "fattorino" quando serve, per questo non serve un wait aggiuntivo all'inizio di scorta_ingrediente.

Esercizio 104)

- Come sappiamo l'indice X dell'architettura di file system FAT-X denota il numero di bit utilizzati per esprimere l'indice di blocco. Per le ipotesi del quesito abbiamo X = 15, il che significa che l'intera partizione potra' constare di 2^{15} = 32.768 blocchi. Poiche' il quesito richiede che tutte le altre caratteristiche dell'architettura in esame restino uguali allo standard FAT-16, i blocchi su disco saranno ampi al max 32 KB = $2^5 \times 2^{10}$ B = 2^{15} B. La dimensione massima di file cosi' come la dimensione massima di partizione saranno dunque fissate a:
  2^{15} blocchi × 2^{15} B / blocco = 2^{30} B = 1 GB,
  ovvero meta' della dimensione max ottenibile con FAT-16.

Esercizio 105)

N_R = numero di *record* che compongono il *file* = 10.000
D_R = dimensione di un *record* = 80 *byte*
D_I = dimensione di un indirizzo = 4 *byte*
D_B = dimensione di un blocco = 512 *byte*

N_{RB} = numero di *record* per blocco = int(D_B/DR) = int(512/80) = int(6,4) = 6
N_{BF} = numero di blocchi occupati dal *file* = 1+int(N_R/N_{RB}) = 1+int(10000/6) = 1.667
N_{IB} = numero di indirizzi in un blocco = D_B/D_I = 512/4 = 128
N_{ID} = numero di indirizzi in X blocchi a doppia indirezione = X^2

I blocchi da indirizzare sono N_{BF} = 1.667

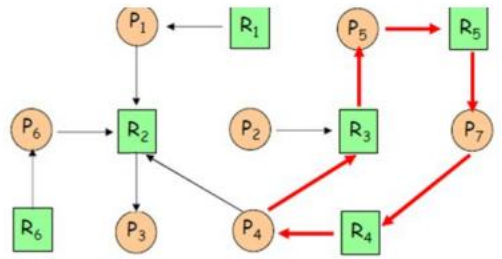
- di questi, 12 possono essere indirizzati direttamente dal nodo indice principale
- dei rimanenti 1.667-12 = 1.655, N_{IB} (cioè 128) sono indirizzabili ad indirezione singola tramite l'indirizzo ad indirezione singola del nodo indice principale
- dei rimanenti 1.655-128 = 1.527, si possono utilizzare blocchi indiretti di 128 con indirezione doppia, come mostrato in figura. → 1 + int1527/128 = 12 blocchi

Con le indicazioni riportate in figura possiamo concludere che:

- per allocare il *file* dati sono necessari NBF blocchi, cioè 1.667 blocchi
- per gestire l'allocazione del *file* sono necessari:
  - 1 blocco per il nodo indice principale
  - 1 blocco di indirizzi ad indirezione singola
  - 1+12 blocchi per l'indirezione doppia

  per un totale di 1+1+1+12 = 15 blocchi
- l'occupazione totale in memoria secondaria vale 1.667 + 15 = 1.682 blocchi da 512 *byte*, per un totale di 1.682 o 512 = 861.184 = *byte* = 841 kB

Esercizio 106)



P4 -> R3 -> P5 -> R5 -> P7 -> R4 -> P4 sono in deadlock

Esercizio 107)

```
void Lettore (void) {
  while (true) {
    P(mutex);
    numeroLettori++;
    if (numeroLettori==1) P(db);
    V(mutex);
    // leggi il dato
    P(mutex);
    numeroLettori--;
    if (numeroLettori==0) V(db);
    V(mutex);
    // usa il dato letto
  }
}
```

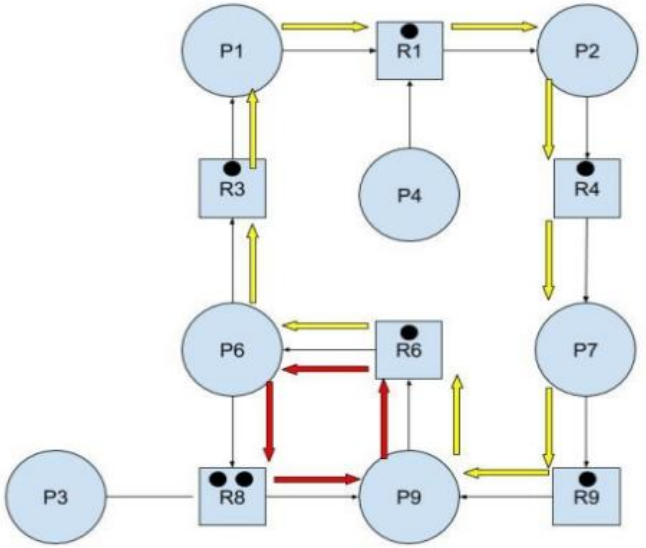
```
void Scrittore (void) {
  while (true) {
    // prepara il dato da scrivere
    P(db);
    // scrivi il dato
    V(db);
  }
}
```

Esercizio 108)

a) nel caso di minima frammentazione, tutti i blocchi dei files sono contigui e disposti uno di seguito all'altro. Si tratta perciò di leggere 100 x 1000 = 100.000 blocchi;
per ognuno servono 100 microsecondi, dunque il risultato è 100.000 x 100 microsecondi = 10 secondi.
(e' considerato corretto anche il risultato che conta 5 ms iniziali per l'accesso al primo blocco: 10 s + 5 ms)

b) nel caso di massima frammentazione, dato che ci sono in tutto 2^31 / 2^12 blocchi = 524.288 blocchi, è possibile considerare che fra ogni blocco di dati ce ne sia uno libero. Pertanto i files sono tutti "polverizzati". Ad ogni lettura va sommato quindi un "salto" (perché non si tratta di blocchi contigui):
tempo totale = 10 secondi (dal punto precedente) + 100000 x 5 ms = 8 minuti e 30 secondi.

Esercizio 109)



Due cicli:
stallo nel ciclo "giallo";
nel ciclo rosso c'è R8 con 2 risorse.

Esercizio 110)

Esistono tante soluzioni

Ordine dipende da quale processo inizia per primo, dunque il primo blocco può essere

8:12

Oppure

1 8:12

Dopo 12: può esserci 2: oppure 13:, in 13 sicuro P2 si blocca,

dunque 2: 13: 3:5 14:17

Oppure 2:5 13:17

Oppure 13: 2:5 14:17

Queste non sono tutte le soluzioni, ricordate che 1: può inserirsi in mezzo all'esecuzione di P2, dunque creando diverso ordine.

Esercizio 111)

32 bit con blocchi da 1KB, dunque 256 indirizzi per blocco

File di dimensione massima usa "tutto"
12 diretti + 256 della singola indirezione +256^2 della doppia = 65804
65804 da 1KB

Spazio occupato dall'inode è 1 (inode principale) + 1 (per la singola indirezione) + (1+256) (per la doppia indirezione)
Dunque 259 KB

Esercizio 112)

Possibili valori (non intendo valori dentro la sequenza, ma i possibili valori finali):
5 7 9 11 13 15
x=x+2 è atomica
Invece nel ThreadA può accadere che legga x, poi eseguo (x=x+2) atomico e poi effettuo incremento x+1; in base al numero di volte che accade varia il valore finale di x

Vediamo un esempio:
eseguiamo 4 volte ThreadB
X=0 -> 2 -> 4 -> 6 -> 8
ThreadA legge 8
ThreadB viene eseguito per ultima volta x -> 10
ThreadA viene eseguito le sue 5 volte, però ThreadA parte ad incrementare x considerando la lettura x=8, dunque 8+5=13

Esercizio 113)

- Notiamo innanzitutto, che, date le dimensione di record e blocco sopra specificate, ogni record del file occupa esattamente un blocco del disco. Quindi:

- allocazione contigua: 1 accesso **alla memoria** di massa permetterà di reperire direttamente il record 8.
- allocazione concatenata: 5 accessi **alla memoria**, in quanto i record che separano il terzo dall'ottavo devono essere letti sequenzialmente.
- allocazione indicizzata: 1 access0 **alla memoria**, (perche' gia' ho avuto accesso al blocco indice).

Esercizio 114)

per ogni blocco (6 msec × 13) + 10 msec + 25 msec,
il 13 è il numero medio di tracce tra i blocchi.

Per il totale moltiplicare per 20 (circa 2.3 secondi)

Esercizio 115)

No, non soddisfa i vincoli necessari (vedi dispense)
(6/9)+(5/15)+(1/5) = 1.2 che non è <=1

Esercizio 116)

No, supponiamo AA, dunque
Primo processo A: X=1; Y=1
Secondo processo A: X=0; Y=2
Poi
Primo processo B: X=1; Y=1 (ho due P(Y) e un V(Y))
Secondo processo B si blocca al 2° P(Y), solo A può andare

Esercizio 117)

Codice errato,
E.g. coke machine è piena, il produttore si blocca su emptyBuffers dopo aver preso il "controllo" di mutex, dunque nessun Consumer può superare il mutex.
Simmetrico, se il consumer esegue mutex e poi si blocca nel fullBuffer.

Esercizio 118)

Raid 0, basta che un disco qualunque sia guasto, dunque
(N° dischi)/MTF 4/1000

Attenzione, segue distribuzione esponenziale, sia: n=n° dischi; min(X) rottura del 1° disco del RAID
P(min(X)<=t) = 1-(1-F(t))^n=1-(e^(-t/MTF))^n ≅ n/MTF (t=1 giorno)

Raid 1, abbiamo 4 dischi: A; B; C; D.
C e D sono mirror di A e B
Dunque perdiamo dati se:
guasto in A e C, oppure guasto in B e D.
P(disco si guasti)=1/MTF=0.001
P(disco si guasti in 10 giorni)=0.01
P(A si guasti e C si guasti in 10 giorni)=10^-5
Però abbiamo due coppie in mirroring, dunque raddoppiare il precedente valore
2×10^-5

Esercizio 119)

semaforo Pala = 1; Semaforo “non riempito” = MAX; Semaforo seme = 0;
semaforo Riempimento = 0;

```
Davide()
While (1)
        Wait(“non riempito”)
      Wait(pala)
        Scavare
        Signal(seme)
        Signal(pala)

Nicola()
While (1)
        Wait(seme)
        Semina
        Signal(riempimento)

Stefano()
While (1)
        Wait(riempimento)
        Wait(pala)
        Riempie
        Signal(“non riempito”)
        Signal(pala)
```

Esercizio 120)

```
Sem  nastro magnetico= 30;
Sem  stampanti= 15;
cliente() {
P(nastro magnetico r);
leggi();
V(nastro magnetico r);
P(stampanti);
stampa();
V(stampanti);
}
```

Esercizio 121)

Ts=1.4
Tl=1000/(10000/60)/2=3
Tt=[(byte da trasferire)/(velocità rotazionale* byte per traccia)]*1000
Abbiamo 8 facce dunque ogni faccia 512/8 GB (2^36)
Numero di byte per traccia=2^36/2^19
Dunque
Tt=1000*[2^15/((10000/60)*2^17)]=1,5ms

Tempo totale = 1,4+3+1,5=5,9
Però abbiamo lettura in parallelo su 8 facce, dunque dividere il tempo di Trasferimento per 8:
Tempo totale = 1,4+3+1,5/8=4,58

Esercizio 122)

Tempo per eseguire operazione di I/O

5+2+0.1=7.1 ms

Dunque ogni disco riesce ad eseguire

1 sec / 7.1 ms = 140 IOPS

140 X 0.7 = 98 IOPS

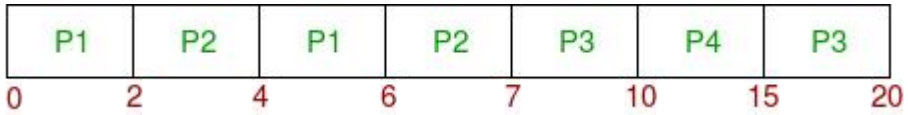
4900/98 = 50 dischi

Con 50 dischi ampiamente soddisfo la richiesta di un TB

Esercizio 123)

300 000 / 1024 = 292.96, dunque 293° blocco
Ogni blocco può contenere 1024/2 indirizzi = 512
Blocchi da 0 a 9 sono diretti
10-521 indiretto primo livello, qua c'è il 293°

Esercizio 124)



Esercizio 125)

Processo W	Processo X	Processo Y	Processo Z
Wait(S)	Wait(S)	Wait(S)	Wait(S)
Read(x)	Read(x)	Read(x)	Read(x)
x=x+1	x=x+1	x=x-2	x=x-2
Write(x)	Write(x)	Write(x)	Write(x)
Signal(S)	Signal(S)	Signal(S)	Signal(S)

x=0;
W legge x=0
Y prende controllo viene eseguito
Z viene eseguito
W ritorna in esecuzione e assegna a x=1 (aveva già letto x=0), così annulliamo il decremento di Y e Z.
Ora tocca a X che incrementa a sua volta x a 2

Esercizio 126)

Avviene deadlock e i valori di (x,y,z) sono (2,1,2).
Prima vengono eseguiti P(lock1) di foo() e P(lock2) di bar(), e.g. bar() esegue P(lock2) prima del P(lock2) di foo(); non importa ordine, poi entrambi i processi sono bloccati, foo() su P(lock2) e bar() su P(lock1)

Esercizio 127)

	Assegnate			Massimo		
	A	B	C	A	B	C
P1	0	1	4	4	1	4
P2	2	0	1	3	1	4
P3	1	2	1	5	7	13
P4	1	0	3	1	1	6

Se assegniamo 6 istanze di C a P3, allora modifichiamo la corrispondente riga di P3 di Assegnate in:

P3	1	2	7

A questo punto calcolo Disponibili (totali-assegnate) e Necessità
Disponibili

A	B	C
1	5	1

Necessità

	A	B	C
P1	4	0	0
P2	1	1	3
P3	4	5	6
P4	0	1	3

Non possono essere soddisfatte le richieste di nessuno dei processi:
1. P1 chiede 4 risorse di tipo A, ma una sola è disponibile;
2. Tutti gli altri processi chiedono >1 risorse C, ma una sola è disponibile.

Stato non sicuro

Esercizio 128)

Ogni volta si accede alla TLB, nel caso di successo accediamo direttamente alla memoria, altrimenti dobbiamo accedere alla tabella delle pagine, dunque:
(200+20)*(HitRatio)+(200+200+20)*(1-HitRatio)=240
220HitRatio+420-420HitRatio=240
-200HitRatio=-180
HitRatio=180/200=0.9

Esercizio 129)

Caso peggiore (risposta "base") tutti in sequenza sullo stesso core, dunque 5+10+20 ms; In realtà caso peggiore potrebbe essere anche tempo infinito, nel caso in cui non vadano mai in run, non ci sono informazioni sufficienti nel testo.
Caso migliore, ogni processo in esecuzione su un core diverso, per il tempo totale di esecuzione basta prendere il massimo dei tre, dunque 20 ms.

Esercizio 130)

200 Gbyte sono ~50 milioni di blocchi, se ogni blocco è 4 Kbyte.
2^25 = 33554432
2^26 = 67108864
Dunque servono almeno 26 bit per gestire partizione massima di 200 Gbyte con blocchi da 4 Kbyte.

Esercizio 131)

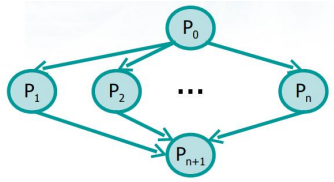
$$S_c^{512} = 1 / (1 - 0.88 \times 0.93) = 5.5$$

$$S_c^{1M} = 1 / (1 - 0.88 \times 0.97) = 6.85$$

$$\frac{T_m}{hT_c + (1-h)T_m} = \frac{1}{1 + h\left(\frac{T_c}{T_m} - 1\right)}$$

$$\frac{S_c^{1M} - S_c^{512}}{S_c^{512}} = 0.24$$

Esercizio 132)




```
init (S1, 0);
init (S2, 0);
```

Process 0

```
...
i=1
while (i<=n) {
  signal (S1);
  i++;
}
...
```

Process i

```
wait (S1);
...
signal (S2);
...
```

Process n+1

```
...
i=1;
while (i<=n) {
  wait (S2);
  i++;
}
...
```

Esercizio 133)

Cerchiamo una sequenza di esecuzione che porti ogni processo al suo completamento:

risorse disponibili

R1	R2	R3	R4
0	3	1	2

allocazione

R1	R2	R3	R4
1	0	2	2
1	2	0	1
2	1	2	0
0	0	0	2

richieste

R1	R2	R3	R4
0	1	0	2
0	4	1	0
0	2	0	5
1	2	1	0

- Cerchiamo una sequenza di esecuzione che porti ogni processo al suo completamento:

risorse disponibili

R1	R2	R3	R4
0	2	1	0

allocazione

R1	R2	R3	R4
1	1	2	4
1	2	0	1
2	1	2	0
0	0	0	2

richieste

R1	R2	R3	R4
0	0	0	0
0	4	1	0
0	2	0	5
1	2	1	0

- Il processo P1 alloca 1 R2 e 2 R4 e termina la sua computazione.

risorse disponibili

R1	R2	R3	R4
1	3	3	4

- Cerchiamo una sequenza di esecuzione che porti ogni processo al suo completamento:

risorse disponibili

R1	R2	R3	R4
0	1	2	4

allocazione

R1	R2	R3	R4
0	0	0	0
1	2	0	1
2	1	2	0
1	2	1	2

richieste

R1	R2	R3	R4
0	0	0	0
0	4	1	0
0	2	0	5
0	0	0	0

- Il processo P4 alloca 1 R1, 2 R2 e 1 R3 e termina la sua computazione.

risorse disponibili

R1	R2	R3	R4
1	3	3	6

- Cerchiamo una sequenza di esecuzione che porti ogni processo al suo completamento:

risorse disponibili

R1	R2	R3	R4
1	1	3	1

allocazione

R1	R2	R3	R4
0	0	0	0
1	2	0	1
2	3	2	5
0	0	0	0

richieste

R1	R2	R3	R4
0	0	0	0
0	4	1	0
0	0	0	0
0	0	0	0

- Il processo P3 alloca 2 R2 e 5 R4 e termina la sua computazione.

risorse disponibili

R1	R2	R3	R4
3	4	5	6

- Cerchiamo una sequenza di esecuzione che porti ogni processo al suo completamento:

risorse disponibili

R1	R2	R3	R4
3	0	4	6

allocazione

R1	R2	R3	R4
0	0	0	0
1	6	1	1
0	0	0	0
0	0	0	0

richieste

R1	R2	R3	R4
0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

- Il processo P2 alloca 4 R2 e 1 R3 e termina la sua computazione.

risorse disponibili

R1	R2	R3	R4
4	6	5	7

Esercizio 134)

- Codice: 200 KB / 16 KB = 12.5 → 13 pagine
- Dati 126 KB / 16 KB = 7.88 → 8 pagine
- Stack: 52 KB / 16 KB = 3.25 → 4 pagine

- 64 MB / 16 KB = 4096 pagine contenute in memoria che richiedono 12 bit per essere indirizzate (2^{12}=4096).

- La tabella delle pagine contiene 25 elementi di 12 bit ciascuno.

Esercizio 135)

- Dimensione pagina: 16 KB = 16384 B
- Numero di pagine residenti in memoria:
  128 KB / 16 KB = 8 pagine
- Codice: 35250 B / 16384 B = 2.15 → 3 pagine
- Dati: 68000 B / 16384 B = 4.15 → 5 pagine
- Stack: 17000 B / 16384 B = 1.04 → 2 pagine

- Il programma necessita di 10 pagine ma la memoria ne può contenere solo 8.

- Dimensione pagina: 8 KB = 8192 B
- Numero di pagine residenti in memoria:
  128 KB / 8 KB = 16 pagine
- Codice: 35250 B / 8192 B = 4.30 → 5 pagine
- Dati: 68000 B / 8192 B = 8.30 → 9 pagine
- Stack: 17000 B / 8192 B = 2.08 → 3 pagine

- Il programma necessita di 17 pagine ma la memoria ne può contenere solo 16.

- Il più lungo programma eseguibile occupa tutto l'indirizzamento logico:

6 per indirizzi (20 bit totali - 14 per usati per offset 2^14 = 16 KB (per indicizzare ogni byte di una data pagina). abbiamo 20 bit, 14 servono per offset allora mi rimangono 6 bit, 2^6 (64) pagine logiche che poi saranno mappate in pagine fisiche.

occupa quindi 64 pagine.

Esercizio 136)

$$(x, y, z) = (3, 3, \{1, 2, 3\})$$

z non è protetto da semafori, dunque le operazioni possono annullarsi a vicenda se vanno bene entrambe allora z=3, se viene sovrascritta quella di foo (da parte di bar) allora z=1, z=2 se invece avviene opposto.

Esercizio 137)

- Il numero n di processi è ottenibile dividendo la dimensione di RAM libera per lo spazio occupato naturalmente da un solo processo.
- $p = 80\%$
- $n = (1GB - 512\ MB) / 128\ MB = 512/128 = 4$
- Utilizzo CPU $= 1 - p^n = 1 - 0.8^4 = 0.59$
- $p = 80\%$
- Nuova RAM: 512MB
- $n = (1GB - 512\ MB + 512\ MB) / 128\ MB = 1024/128 = 8$
- Utilizzo CPU $= 1 - p^n = 1 - 0.8^8 = 0.8322$

Esercizio 138)

- Dimensione pagina: 4 KB = 4096 B
- Numero di pagine residenti in memoria:
  128 KB / 4 KB = 32 pagine
- Codice: 35250 B / 4096 B = 8.61 → 9 pagine
- Dati: 68000 B / 4096 B = 16.60 → 17 pagine
- Stack: 17000 B / 4096 B = 4.15 → 5 pagine

- Il programma necessita di 31 pagine e la memoria ne può contenere 32, quindi il programma può essere eseguito rimanendo completamente in memoria.

- Periodo: tempo entro il quale l'evento deve essere ripetuto (Pi)
- Tempo d'esecuzione (Ci)
- Sistema Real Time sostenibile (con m eventi periodici): vale

$$\sum_{i=1}^{m} \frac{C_i}{P_i} \leq 1$$

Procediamo con la sostituzione dei valori all'interno della formula:

$$\frac{C_0}{P_0} + \frac{C_1}{P_1} + \frac{C_2}{P_2} + \frac{C_3}{P_3} \leq 1$$

Sostituendo in modo opportuno si ottiene:

$$\frac{100}{500} + \frac{100}{300} + \frac{3}{15} + \frac{200}{800} = \frac{59}{60} \leq 1$$

Massimo tempo di esecuzione per il nuovo processo P_4.

$$\frac{C_0}{P_0} + \frac{C_1}{P_1} + \frac{C_2}{P_2} + \frac{C_3}{P_3} + \frac{C_4}{P_4} \leq 1$$

Dato P_4 e C_4 sostituire nuovamente i valori e verificare il risultato.

$$\frac{100}{500} + \frac{100}{300} + \frac{3}{15} + \frac{200}{800} + \frac{2}{4} = \frac{89}{60} \leq 1$$

Ma attenzione perchè:

$$\frac{89}{60} \not\leq 1$$

Si può concludere quindi che il sistema non rimane sostenibile.

Esercizio 139)

- **First-fit:** inserisce il nuovo processo nella prima posizione possibile dall'inizio della lista.
- Best-fit: inserisce il nuovo processo nella posizione tale da contenerlo nel modo migliore e, dunque, nel buco più piccolo disponibile per contenere il processo.

Primo spazio sufficiente per contenere il processo inizia all'inidirizzo 6. Si ottiene:

- P,0,6,→P,6,2,→H,8,1,→ P,9,8→P,17,4,→H,21,2,→P,23,6,→H,29,4,X

Miglior spazio per contenere il processo inizia all'inidirizzo 21. Si ottiene:

- P,0,6,→H,6,3,→P,9,8→P,17,4,→P,21,2,→P,23,6,→H,29,4,X

Esercizio 140)

execution time = number of instruction used × (cycle per instruction + memory reference per instruction × miss rate × miss penalty) × cycle time

con cache: noi*(2+1.33*0.02*50)*ct=3.33*(ct*noi)
senza cache: noi*(2+1.33*1*50)*ct=68.5*(ct*noi)
nel caso di cache perfetta (nessun miss): noi*(2+1.33*0*50)*ct=2*(ct*noi)

Esercizio 141)

- 64 MB di memoria richiedono 26 bit di indirizzamento (2^{26} = 64 MB).
- 64 KB di memoria richiedono 16 bit di indirizzamento (2^{16} = 64 KB).
- Sono necessari 7 registri di 42 bit ciascuno.

- Il più lungo programma eseguibile occupa tutto l'indirizzamento logico:
- 2^{20} B / 64 KB = 1 MB /64 KB = 16 segmenti. (Oppure 20 bit – 16 bit = 4 bit → 2^4 = 16 segmenti).
- Il più lungo programma eseguibile necessita di 16 registri di segmentazione.

Esercizio 142)

- la soluzione proposta fa si che
  - la (N+1)-ma auto va via
  - l'autobotte va via se c'e' almeno un'auto

```
sem distributore  // semaforo inizializzato a 1
int occupate = 0 // intero condiviso inizializzato a 0
sem serbatoio // semaforo inizializzato a 1
int disponibile = M // intero condiviso inizializzato a M

void autobotte ( )
wait (distributore)
if ( occupate > 0)
   signal (distributore)
else
   riempi ( )
   disponibile = M
   signal (distributore)
endif
end autobotte
```

autobotte usa "distributore" per mutua esclusione per modificare 'disponibile' e non "serbatoio" come più logico dal nome (come nella soluzione di automobile()), ma non è un problema, perché tanto autobotte entra solo se non ci sono auto in attesa, dunque funziona con tale vincolo.

```
void automobile ( )
wait ( distributore )
if ( occupate < N )
    occupate = occupate + 1
    signal ( distributore )
    richiesta = rand (1 .. 20)
    wait (serbatoio)
    if (richiesta > disponibile )
          signal (serbatoio)
    else
          disponibile = disponibile - richiesta
          signal (serbatoio)
          faibenzina( )
    endif
    wait(distributore)
    occupate = occupate -1
    signal(distributore)
else
    signal (distributore)
endif
end automobile
```

Esercizio 143)

```
int n1 = 0;
int n2 = 0;
```

s	t
s_1: **while**(n1 <= 500) {	t_1: **while**(n2<=500) {
s_2: n1 = n1 + 1;	t_2: n2 = n2 + 1;
s_3: n2 = unknown(n1,n2)	}
}	

Numero di iterazioni di s() è sempre 501.
Caso massimo per t, arrivo a n2=501 (poi uscirei dal while), però prima del controllo del while() eseguo s() una prima volta, n2 posto a n1 (dunque 1). Riparto con t(), n2 fino a 501 poi eseguo s e pongo n2 uguale a 2, così via fino al termine di s().
$\sum_{k=0}^{500}(501-k)$

Esercizio 144)

Inizialmente S_1 vale 0 e S_2 vale 0

A:
```
while(TRUE){
  opA;
  signal(S1);
  wait(S2);
}
```

B:
```
while(TRUE){
  wait(S1);
  opB;
  signal(S2);
}
```

Esercizio 145)

- Caso $C = T$. $SB/B = \frac{C+T}{\max\{T,C\}} = \frac{T+T}{T} = 2$.
- Caso $C > T$. $SB/B = \frac{C+T}{\max\{T,C\}} = \frac{C+T}{C} < \frac{C+C}{C} = 2$.
- Caso $C < T$. $SB/B = \frac{C+T}{\max\{T,C\}} = \frac{C+T}{T} < \frac{T+T}{T} = 2$.

Esercizio 146)

```
22 release(int number) { # < free = free + number; >
23
24   P(enter);
25   free := free + number;
26   if (cnt > 0) {        # Some process is waiting
27     V(delay);           # Give them priority
28   }
29   else V(enter);        # Else: open for new users
30 }
```

Esercizio 147)

- occupazione bitmap > occupazione lista se e solo se
  S(T-U) < T cioe' U > (ST-T)/S

Esercizio 148)



Blocchi distribuiti per la correzione di errori
Perdita di dati se guasti due dischi qualunque

Prob. che un fissato disco (ad es. A) si guasti = 1/MTF = 0.001

Prob. che si guastino
- A e B nell'arco di 10 giorni = 0.00001
- A e C nell'arco di 10 giorni = 0.00001
- A e D nell'arco di 10 giorni = 0.00001

} 3*0.00001 = 0.00003

(in generale (D-1)*MTR*MTF²)

Prob. che si guastino nell'arco di 10 giorni
- B e C oppure B e D oppure C e D 0.00003

una qualunque coppia di dischi = 0.00006
(in generale D*(D-1)*MTR/(2*MTF²))

Esercizio 149)

$0.75 \times (200 \text{ nanoseconds}) + 0.25 \times (400 \text{ nanoseconds}) = 250 \text{ nanoseconds}.$

Esercizio 150)

$$\begin{aligned} 0.2\ \mu\text{sec} &= (1-P) \times 0.1\ \mu\text{sec} + (0.3P) \times 8 \text{ millisec} + (0.7P) \times 20 \text{ millisec} \\ 0.1 &= -0.1P + 2400\,P + 14000\,P \\ 0.1 &\simeq 16{,}400\,P \\ P &\simeq 0.000006 \end{aligned}$$

Esercizio 151)

nel caso peggiore, per ogni quanto associato al CPU bound ci sarà un nuovo processo I/O bound al quale assegnare il quanto

- The time quantum is 1 millisecond: Irrespective of which process is scheduled, the scheduler incurs a 0.1 millisecond context-switching cost for every context-switch. This results in a CPU utilization of 1/1.1 * 100 = 91%.
- The time quantum is 10 milliseconds: The I/O-bound tasks incur a context switch after using up only 1 millisecond of the time quantum. The time required to cycle through all the processes is therefore 10*1.1 + 10.1 (as each I/O-bound task executes for 1 millisecond and then incur the context switch task, whereas the CPU-bound task executes for 10 milliseconds before incurring a context switch). The CPU utilization is therefore 20/21.1 * 100 = 94%.

Esercizio 152)

a) FCFS, perché aumento priorità del processo in esecuzione, dunque quello in esecuzione rimarrà nel core finché non ha finito esecuzione, dunque otteniamo un FCFS. Ricorda che i processi nella coda pronti aumentano anche loro priorità in base da quanto sono nella coda.

b) LIFO con prelazione, il processo in esecuzione ha priorità che cala e dunque è inferiore al processo appena arrivato nella coda pronti.

Esercizio 153)

```
monitor file_access {
int curr_sum = 0;
int n;
condition c;

void access_file(int my_num) {
while (curr_sum + my_num >= n)
    c.wait();
curr_sum += my_num;
}

void finish_access(int my_num) {
curr_sum -= my_num;
c.broadcast();
}
}
```

Esercizio 154)

(0.8)*(1)+0.1*(1+1)+0.1*(25000+25000+1+1)
~ 5 ms

Esercizio 155)

Yes, a RAID Level 1 organization could achieve better performance for read requests. When a read operation is performed, a RAID Level 1 system can decide which of the two copies of the block should be accessed to satisfy the request. This choice could be based on the current location of the disk head and could therefore result in performance optimizations by choosing a disk head which is closer to the target data.

Esercizio 156)

```
class CandyShop {
   public:
     void Enter(); void Finish();
   private:
     int ticket;
     Semaphore mutex;    // control access to shared variables
     Semaphore servers; // Sales people
}
CandyShop::CandyShop(int n) {
     ticket = 0;
     mutex.value = 1;
     servers.value = n;  // Number of Sales people
}
CandyShop::Enter() {
   mutex.Wait();
   ticket++;
   mutex.Signal();
   servers.Wait();
}
CandyShop:: Finish() {
   servers.Signal();
}
```

Attenzione con questa soluzione non si considera il numero di ticket per l'ordine dei clienti serviti. servers.wait può essere eseguito da altro processo con ticket maggiore, i.e. P1 esegue mutex.signal, P2 incrementa ticket e poi può succedere che esegua servers.wait prima di P1. Modo più semplice è aggiungere altra variabile condivisa che controlli numero ticket prossimo cliente

Esercizio 157)

One thread. For two threads for example, we can get deadlock with the following:

Example

```
Thread 1:
   a.Wait();
   b.Wait();
   ...
```

```
Thread 2:
   b.Wait();
   a.Wait();
```

Esercizio 158)

```
monitor alarm {
   condition c;

   void delay(int ticks) {
     int begin_time = read_clock();
     while (read_clock() < begin_time + ticks)
     c.wait();
   }

   void tick() {
     c.broadcast();
   }
}
```

Esercizio 159)

```
monitor resources
{
    int available_resources;
    condition resources_avail;

    int decrease_count(int count)
    {
        while (available_resources < count)
    resources_avail.wait();
        available_resources -= count;
    }

    int increase_count(int count)
    {
        available_resources += count;
        resources_avail.broadcast()
    }
```

Esercizio 160)

Indirizzi a 32 bit e pagine da 4 KB (2^{12}). Dunque ogni pagina contiene 1024 indirizzi, la prima metà (dunque 2^9) sono indirizzi diretti, la 2° metà indicizza delle tabelle delle pagine (dunque 2^9 tabelle), sommando:

$$(2^9 + 2^{19}) * 2^{12} = 2^{21} + 2^{31}\ bytes$$

Esercizio 161)

bits 3, 5, and 6 are data bits and the remaining bits are parity bits, and that we are using odd parity.

$$\underset{1}{0}\ \underset{2}{0}\ \underset{3}{1}\ \underset{4}{1}\ \underset{5}{1}\ \underset{6}{0}$$

Bits 3 and 5 are 11, so bit 1 should be 1 for odd parity.
Bits 3 and 6 are 10, so bit 2 is correct
Bits 5 and 6 are 10, so bit 4 should be 0.

Add the numbers of the parity bit positions that are incorrect. $1 + 4 = 5$, so data bit 5 is incorrect. The correct code is 001100, corresponding to data 100 .

Esercizio 162)

```
semaphore S[3:6] = ([4] 0); // init
T1: …signal(S[3]); signal(S[4]);
T2: …signal(S[3]); signal(S[4]);
T3: wait(S[3]); wait(S[3])… signal(S[5]); signal(s[5]);
T4: wait(S[4]); wait(S[4])… signal(S[6]);
T5: wait(S[5]); … signal(S[3]);
T6: wait(S[6]); … signal(S[3]);
T7: wait(S[5]); wait(S[3]); wait(S[3]);…
```

Esercizio 163)

Il semaforo s non influisce sul flusso di esecuzione e quindi l'output dipende dall'ordine di esecuzione di R1–R2 e R3–R4:

- Prima R1, R3 (in qualsiasi ordine) e poi R2, R4 (in qualsiasi ordine) → stampa: 10 10
- R1 R2 R3 R4 → stampa: 11 10
- R3 R4 R1 R2 → stampa: 9 10

Esercizio 164)

Each datablock will have its entry.

So, Total Number of entries in the FAT = $\frac{\text{Disk Capacity}}{\text{Block size}} = \frac{100MB}{1KB} = 100K$

Each entry takes up $4B$ as overhead

So, space occupied by overhead = $100K \times 4B = 400KB = 0.4MB$

We have to give space to Overheads on the same file system and at the rest available space we can store data.

So, assuming that we use all available storage space to store a single file = Maximum file size = $\text{Total File System size} - \text{Overhead} = 100MB - 0.4MB = 99.6MB$

Esercizio 165) 2<=tally<=100;

100 se non c'è interferenza

2 se:
Il processo A carica il valore di tally, incrementa tally, ma poi perde il processore (ha incrementato il suo registro a 1, ma non ha ancora memorizzato questo valore).
Il processo B carica inizialmente il valore di tally, che è zero. Quindi esegue quarantanove operazioni di incremento prima di perdere il controllo del processore, con il risultato che il valore 49 viene memorizzato nella variabile condivisa tally.
Il processo A riprende momentaneamente il controllo per eseguire la sua prima operazione di memorizzazione, sostituendo il precedente valore di tally di 49 con 1. Tuttavia, è costretto a rinunciare immediatamente al processore.
Il processo B riprende, caricando il valore corrente di tally (1) nel suo registro prima di essere costretto a rinunciare al processore. Si noti che questo era l'ultimo caricamento del processo B.
Il processo A viene eseguito fino al completamento senza interruzioni, eseguendo le restanti 49 operazioni di caricamento, incremento e memorizzazione, impostando infine il valore di tally a 50.
Il processo B viene riattivato con una sola operazione di incremento e memorizzazione prima di terminare. Incrementa il valore del suo registro a 2 e lo memorizza come valore finale della variabile condivisa.

Esercizio 166) Per calcolare il numero di bit necessari per specificare una pagina possiamo usare la formula 2^32 /2^10 = 2^22 pagine, 22 bit per specificare una pagina. Ogni tabella di pagina contiene 2^10 byte / (4 byte per voce) = 2^8 voci. In questo modo la tabella può gestire 8 bit dei 22 necessari per indirizzare ogni pagina. Con 8 bit per tabella di pagina, sono necessari 3 livelli di tabelle di pagina.

PS offset è di 10 bit, dunque 32-22 bit per gestire pagine

Esercizio 167)

Con s=2 e T=0, l'insieme di possibili output è ·

In un caso (i.e. valori di x dopo esecuzione) i valori saranno 6 o 10, nel caso di considerare di write(x) saranno 5, 6, 10.

Esercizio 168)

```
semaphore accesso = 1; /* forza l'ordinamento delle macchine che arrivano */
semaphore portata = 3; /* capacita' del ponte */
semaphore mutexsx = 1; /* per controllo di flusso e mutex */
semaphore mutexdx = 1; /* per controllo di flusso e mutex */
int nsx = 0; /* macchine che hanno ottenuto accesso da sx */
int ndx = 0; /* macchine che hanno ottenuto accesso da dx */

macchina_dalla_riva_sinistra() {
  wait(accesso); /* inizio protocollo di accesso */
  wait(mutexsx); /* per evitare scontri e per mutex su nsx e ndx */
  ++nsx;
  if (nsx==1) /* prima di un gruppo in questa direzione */
     wait(mutexdx); /* blocca macchine dall'altra riva */
  signal(mutexsx);
  signal(accesso); /* fine protocollo di accesso */

  wait(portata);
  passa_ponte();
  signal(portata);

  wait(mutexsx); /* mutex su nsx e ndx */
  --nsx;
  if(nsx==0) /* ultima di un gruppo in questa direzione */
     signal(mutexdx); /* sblocca macchine dall'altra riva */
  signal(mutexsx);
}
```

Notate che in questa soluzione viene gestito lo starvation. Appena arriva una auto da destra che si blocca sul suo semaforo mutex, comunque ha effettuato il wait accesso. Dunque nessun'altra macchina da sinistra potrà entrare nella regione gestita dal semaforo accesso.
Le macchine a sinistra che sono già entrate nel ponte usciranno, l'ultima sveglierà auto da destra e inizierà il flusso delle auto da destra.

```
macchina_dalla_riva_destra() {
  wait(accesso); /* inizio protocollo di accesso */
  wait(mutexdx); /* per evitare scontri e per mutex su nsx e ndx */
  ++ndx;
  if (ndx==1) /* prima di un gruppo in questa direzione */
     wait(mutexsx); /* blocca macchine dall'altra riva */
  signal(mutexdx);
  signal(accesso); /* fine protocollo di accesso */

  wait(portata);
  passa_ponte();
  signal(portata);

  wait(mutexdx); /* mutex su nsx e ndx */
  --ndx;
  if(ndx==0) /* ultima di un gruppo in questa direzione */
    signal(mutexsx); /* sblocca macchine dall'altra riva */
  signal(mutexdx);
}
```

Esercizio 169)

turn=0; P(1) pone blocked(1) a true, legge blocked(0) con valore false e rimane dentro il while; P(0) assegna a blocked(0) valore true, legge turn=0 ed entra nella sezione critica, P(1) pone turn a 1 ed entra anche lui nella sezione critica.

Esercizio 170)

S porzione seriale
N core

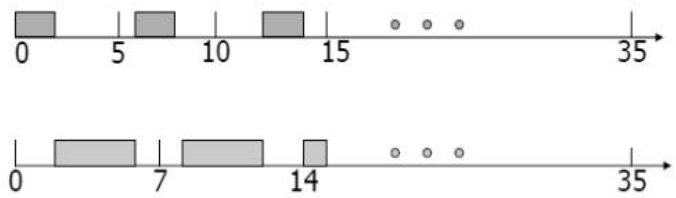
$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

per $N \to \infty$ lo speedup tende a $1/S$

Se il 40% di un'applicazione è seriale il massimo aumento di velocità è di 2.5 volte

Esercizio 171)

Task set: {(2,5),(4,7)}
U = 2/5 + 4/7= 34/35 ~ 0.97 (schedulable!)



Esercizio 172)

Il comportamento non è corretto dato che il protocollo per l'accesso ai dati condivisi (entry–exit section) non viene rispettato

Sequenza scorretta

- p1, p2, c1, c2, c1: attenzione, il consumatore consuma da un buffer vuoto!

Esercizio 173)

Calcolo dell matrice Need

$$\text{Need} = \begin{pmatrix} 0\,1\,0\,0\,2 \\ 0\,2\,1\,0\,0 \\ 1\,0\,3\,0\,0 \\ 0\,0\,1\,1\,1 \end{pmatrix} \qquad \text{Available} = (0,0,x,1,1)$$

Nello stato corrente, può essere eseguito solo il thread T_D, ponendo x=1. T_D restituisce quindi le risorse che aveva allocate e Available diviene (1,1,x+1,2,1).

Ora, solo il thread T_C può ricevere tutte le risorse necessarie, nell'ipotesi però che x=2. Quindi Available si aggiorna a (2,2,x+1,3,1).

Si può servire T_B, senza vincoli ulteriori sul valore di x. Il vettore Available diviene (4,2,x+2,4,2), sufficiente per T_A

Quindi $<T_D, T_C, T_B, T_A>$, con x=2.

Esercizio 174)

La tabella delle pagine invertita contiene 64 elementi ciascuno dei quali contiene un numero di pagina logica (7 bit) ed un identificativo di processo (9 bit)
→ 64×2byte = 128 byte

Esercizio 175)

La pagina che contiene il codice e la variabile `i` è residente in memoria:

- Per 2 pagine contenenti 100 parole per ognuno dei vettori `a` e `b` si hanno 20 page fault;
- Per 3 pagine contenenti 100 parole per ognuno dei vettori `a` e `b` si hanno 20 page fault;
- Per 2 pagine contenenti 500 parole per ognuno dei vettori `a` e `b` si hanno 4 page fault.

Esercizio 176)

Nel caso di allocazione indicizzata, in ogni blocco indice sono contenuti 1024 puntatori, per cui la dimensione massima del file è $(2^{10}\times 2^{10}\times 2^{11})B = 2^{31}B = 2GB$.

blocco sono 2KB, ogni indirizzo 16 bit (2 B), dunque ci stanno 2KB/2B = 1024 (2^10) indirizzi in ogni blocco. Il testo dice indicizzata a due livelli, dunque il primo blocco punta a 2^10 blocchi, ognuno di questi contiene 2^10 indirizzi, dunque abbiamo (2^10)*(2^10) blocchi da 2 KB (2^11 B).

Esercizio 177)

possibili flussi sono:
A1 A2 B1 B2: X=11
B1 B2 A1 A2: X=12

Esercizio 178) semplicemente effettuare switching fra i due cicli for (esterno diventa interno e viceversa)

Esercizio 179)

basta invertire ordine, e.g. modificare Z, prima acquisisce A e poi C.

Esercizio 180) il tempo effettivo sarà 1h+5(1-h), dove h è hit rate, vogliamo che tempo sia inferiore a 2, dunque risolvendo disequazione otteniamo che h deve essere almeno di 0.75

Esercizio 181) per ogni istruzione dobbiamo aggiungere un overhead di n/k, dunque tempo medio diventa 1+n/k

Esercizio 182)

```
sema gate     = 1
sema outgoing = 1
sema empty    = 10
sema occupied = 0
```

```
Supplier
{
  while (TRUE)
  {
    P(gate);
    <Pass through gate>;
    V(gate);

    <Enter incoming
    goods section>;

    P(empty);
    P(empty);
    P(empty);
    <Unload 3 packets>;
    V(occupied);
    V(occupied);
    V(occupied);

    <Leave incoming
    goods section>;

    P(gate);
    <Pass through gate>;
    V(gate);
    }
}
```

```
Deliverer_X
{
  while (TRUE)
  {
    P(gate);
    <Pass through gate>;
    V(gate);

    P(outgoing);
    <Enter outgoing
    goods section>;

    P(occupied);
    P(occupied);
    <Pick 2 packets>;
    V(empty);
    V(empty);

    <Leave outgoing
    goods section>;
    V(outgoing);

    P(gate);
    <Pass through gate>;
    V(gate);
    }
}
```

```
Deliverer_Y
{
  while (TRUE)
  {
    P(gate);
    <Pass through gate>;
    V(gate);

    P(outgoing);
    <Enter outgoing
    goods section>;

    P(occupied);
    <Pick 1 packet>;
    V(empty);

    <Leave outgoing
    goods section>;
    V(outgoing);

    P(gate);
    <Pass through gate>;
    V(gate);
    }
}
```

Esercizio 183)

se foo() eseguisse semWait(S) e poi bar() eseguisse semWait(R), entrambi i processi si bloccheranno quando ciascuno di essi eseguirà la propria istruzione successiva.

Non abbiamo informazioni su priorità ed algoritmo di scheduling, un problema potrebbe essere in tale parte, i.e. sempre in esecuzione uno dei due processi. Non avendo tali informazione, presupponiamo scheduling fair, allora la risposta è 'No'. Se uno dei due processi si bloccasse su una chiamata semWait() allora o anche l'altro processo si bloccherà come descritto in a) o l'altro processo sta eseguendo la sua sezione critica. In quest'ultimo caso, quando il processo in esecuzione esce dalla sua sezione critica, eseguirà una chiamata semSignal() che risveglierà il processo bloccato.

Esercizio 184)

I bit totali sono 7, numerati da 1 a 7.
I bit di parità sono collocati nelle posizioni che corrispondono a potenze di 2: posizioni 1, 2 e 4.
I bit di dati occupano le posizioni 3, 5, 6 e 7.

P_1 (posizione 1): controlla i bit dati nelle posizioni 3, 5 e 7.
P_2 (posizione 2): controlla i bit dati nelle posizioni 3, 6 e 7.
P_3 (posizione 4): controlla i bit dati nelle posizioni 5, 6 e 7.

Nel codice di Hamming a parità dispari, per ciascun gruppo la somma (in termini di "1") incluso il bit di parità deve essere dispari.
La parola ricevuta è 1 0 0 1 1 1 1
dunque:
- P_1 (posizione 1): Corretto → 0
- P_2 (posizione 2): Errato → 1
- P_3 (posizione 4): Errato → 1

L'errore si trova nella posizione 6 (2+4). Quindi la sequenza corretta è:
1 0 0 1 1 0 1
I bit dati sono 0 1 0 1 tale valore è 5, dunque un multiplo di 5

Esercizio 185)

Esattamente 3 'C'. Ogni volta che il Processo 1 esegue l'istruzione "wait(U)", il valore del semaforo U viene decrementato di 1. Poiché non ci sono istruzioni "signal(U)", il ciclo nel Processo 1 verrà eseguito solo 3 volte (cioè, il valore iniziale di U) e poi si bloccherà alla quarta esecuzione della "wait(U)".
Esattamente 3 'D'. Il Processo 1 eseguirà il suo ciclo tre volte (vedi la risposta precedente), incrementando "signal(V)" ogni volta nel ciclo. Questo permetterà alla "wait(V)" di completarsi tre volte. Per ogni "wait(V)" eseguita dal Processo 2, esso esegue anche una "signal(V)", quindi non c'è alcun cambiamento nel valore del semaforo V causato dal Processo 2. Il Processo 3 invece decrementa il valore del semaforo V, scrivendo "D". Quindi il Processo 3 eseguirà alla fine il ciclo tante volte quante il Processo 1.

Esercizio 186)

```
semaphore sync = 1;

Process A
    wait(sync);
A1: tempA = counter + 1;
A2: counter = tempA;
    signal(sync);

Process B
    wait(sync);
B1: tempB = counter + 2;
B2: counter = tempB;
    signal(sync);
```

Esercizio 187)

Un qualsiasi ordine di c1, c2 e c3 (in base se finiscono entro il timeout)

Esercizio 188) stesso ragionamento di ese 181)

Pertanto, il tempo medio per esecuzione è:

$$T_{\text{medio}} = t + \frac{n_1}{k_1} + \frac{n_2}{k_2}$$

Applicazione al caso specifico:

Con $t = 2\,\text{ns}$, $k_1 = 5$, $n_1 = 3\,\text{ns}$, $k_2 = 10$ e $n_2 = 8\,\text{ns}$:

$$T_{\text{medio}} = 2 + \frac{3}{5} + \frac{8}{10} = 2 + 0.6 + 0.8 = 3.4\,\text{ns}$$

Esercizio 189)

Il valore finale di X è sempre 11 in tutti i casi.
Il valore finale di W può essere 0 oppure -6

Esercizio 190)

essendo interi 13/2=6

i valori di X sono sempre 10, mentre W può risultare 6 oppure 0

Esercizio 191)

a. Probabilità di esattamente 2 guasti:

Utilizzando la distribuzione binomiale:

$$P(2\text{ guasti}) = \binom{3}{2} \times p^2 \times (1-p) = 3 \times (0.1)^2 \times (0.9) = 3 \times 0.01 \times 0.9 = 0.027$$

b. Probabilità di esattamente 3 guasti:

$$P(3\text{ guasti}) = \binom{3}{3} \times p^3 = 1 \times (0.1)^3 = 0.001$$

Probabilità totale di guasto per RAID 5:

$$P(\text{guasto RAID 5}) = P(2\text{ guasti}) + P(3\text{ guasti}) = 0.027 + 0.001 = 0.028 \quad (2.8\%)$$

Esercizio 192) Opzionale GOlang

```
package main

import (
"fmt"
"sync"
)

func squareNumber(in <-chan int, out chan<- int, wg *sync.WaitGroup) {
num := <-in
out <- num * num
wg.Done()
}

func main() {
in := make(chan int, 2)
out := make(chan int, 2)

var num1, num2 int
fmt.Print("Inserisci il primo numero: ")
fmt.Scan(&num1)
fmt.Print("Inserisci il secondo numero: ")
fmt.Scan(&num2)

in <- num1
in <- num2

var wg sync.WaitGroup
wg.Add(2)

go squareNumber(in, out, &wg)
go squareNumber(in, out, &wg)

wg.Wait()
close(out)

sum := 0
for val := range out {
sum += val
}

fmt.Println("Somma dei quadrati:", sum)
}
```

Esercizio 193)

Nel caso in cui lo scheduler sia SRTN, il sistema puo' prelazionare ogni processo correntemente in CPU non appena il software di sistema riprende il controllo. Nello scenario descritto questo avviene all'istante T0+3, quando P0 richiede (al sistema) l'arttivazione di P1. Data la regola di priorita' di SRTN, al tempo T0+3 il "remaining time" dei due processi e' 7 per P0 e 6 per P1. P0 viene effettivamente prelazionato e la CPU viene ceduta per 6 secondi a P1. Il controllo della CPU ritorna a P0 al tempo T0+3+6, e P0 terminera' quindi al tempo T0+3+6+7 = T0+16

Esercizio 194) Shortest Remaining Time First

Esercizio 195) 0: se il Processo 3 viene pianificato immediatamente dopo ogni esecuzione di "signal(V)" da parte del Processo 1, il Processo 2 potrebbe continuare a bloccarsi nella sua istruzione "wait(V)" e quindi non eseguire mai le sue istruzioni "type". Finchè Processo 1 sarà bloccato su Wait(U) e tutti i processi sono fermi

Esercizio 196)

SCRIVI:
wait (SB);
<post data to bottom half of M>
signal (INV);

SCRIVI-IMVERTI:
wait (SA);
<post data to top half of M>
wait (INV);
<revert byte order in M>;
signal (SC);

LEGGI:
wait(SC);
<read data from M>;
signal(SA);
signal(SB);

Esercizio 197) Il problema è nel ciclo che non esiste in FAT.
Strategie di correzione:
Interrompere la catena nel primo punto dove il ciclo si chiude:
es. impostare FAT[7] = EOF.

Esercizio 198)

$$T(p) = p \cdot t_C + (1 - p) \cdot t_R$$

dove:

- p è la probabilità che l'accesso sia soddisfatto dalla cache,
- $t_C = 2$ ns (cache),
- $t_R = 50$ ns (RAM).

La probabilità di cache hit varia nel tempo secondo $p(t) = \frac{t}{1+t}$. $\frac{1}{10} \int_0^{10} T(p(t))\, dt$

T(p)=2p+(1-p)50=2p-50p+50=-48(p)+50=
-48(t/(1+t))+50= (-48t+50t+50)/(1+t)=(2t+50)/(1+t)

$$\int_0^{10} \frac{2t + 50}{1 + t} dt$$

$$2t + 50 = 2(t + 1) + 48 \Rightarrow \int_0^{10} \left[2 + \frac{48}{1 + t}\right] dt = 2t + 48 \ln(1 + t)$$

$$\int_0^{10} T(t)dt = 2 \cdot 10 + 48 \ln(11) - (0 + 48 \ln(1)) = 20 + 48 \ln(11)$$

$$\text{Media} = \frac{1}{10}(20 + 48 \ln(11)) \approx \frac{1}{10}(20 + 48 \cdot 2.3979) \approx 13.5 \text{ ns}$$

Esercizio 199)
e.g.:
Worker 2 started
Worker 1 started
Worker 3 started
All workers done

Oppure qualsiasi altra permutazione dei messaggi dei worker.
La funzione principale aspetta che tutti i worker abbiano terminato.
L'ordine è non deterministico a causa della concorrenza.

Esercizio 200)

```
// Inizializzazione
s1 = 0
s2 = 0
s3 = 0
s4 = 0
// P1
BEGIN
  // corpo di P1
  signal(s1)
  signal(s1)   // secondo segnale per sbloccare entrambi
END

// P2
BEGIN
  wait(s1)
  // corpo di P2
  signal(s2)
  signal(s3)
END
```

```
// P3
BEGIN
  wait(s1)
  // corpo di P3
  signal(s2)
  signal(s3)
END

// P4
BEGIN
  wait(s2)   // attende che P2 e P3 segnalino
  wait(s2)   // secondo wait per coprire entrambe le segnalazioni
  // corpo di P4
END

// P5
BEGIN
  wait(s3)
  wait(s3)
  // corpo di P5
  signal(s4)
END

// P6
BEGIN
  wait(s4)
  // corpo di P6
END
```

Esercizio 201)

dal testo sappiamo che:
18=x+y*70
1=x+y
dunque:
y=1-x
18=x+(1-x)*70
18=x+70-70x
18=-69x+70
69x=70-18
x=~0.754
y=~0.246

Esercizio 202)

$$T(2) = 0{,}30 + \frac{0{,}40+0{,}02\cdot 1}{2} + \frac{0{,}30}{2} + 0{,}05 \cdot (1 - \tfrac{1}{2})$$
$$= 0{,}30 + 0{,}21 + 0{,}15 + 0{,}025 = 0{,}685$$

Limite per $p \to \infty$

- $\frac{A+0{,}02\log_2 p}{p} \to 0$
- $\frac{B}{\min(p,16)} \to \frac{B}{16} = \frac{0{,}30}{16} = 0{,}01875$
- $0{,}05(1 - \frac{1}{p}) \to 0{,}05$

Quindi

$$T(\infty) = S + \frac{B}{16} + 0{,}05 = 0{,}30 + 0{,}01875 + 0{,}05 = 0{,}36875$$

unica componente non banale è legata alla parte del code parallelizzabile all’infinito, vedi pagina successiva

Considera

$$f(p) = \frac{A + 0{,}02\,\log_2 p}{p}, \qquad p \to \infty.$$

Scriviamo $\log_2 p = \dfrac{\ln p}{\ln 2}$; allora

Recall: derivata di ln(x) è uguale al reciproco di x; derivata di $\log_a(x)=1/(x*\ln(a))$; dunque semplifico per utilizzare ln(x)

$$f(p) = \frac{A}{p} + \frac{0{,}02}{\ln 2} \cdot \frac{\ln p}{p}.$$

- $\dfrac{A}{p} \xrightarrow[p\to\infty]{} 0.$
- Per l'altro termine, si può usare L'Hôpital:

$$\lim_{p\to\infty} \frac{\ln p}{p} \overset{\text{L'Hôpital}}{=} \lim_{p\to\infty} \frac{1/p}{1} = \lim_{p\to\infty} \frac{1}{p} = 0.$$

(oppure richiamare che $\ln p = o(p)$).

Quindi entrambe le parti vanno a zero e

$$\boxed{\lim_{p\to\infty} \frac{A + 0{,}02\,\log_2 p}{p} = 0.}$$

Esercizio 203)

- Stato iniziale: `(U,V,W) = (3,1,0)`.

1. **P1** ×3: stampa `C C C`, stato `(0,4,3)`.
   (Ogni P1: `U-1, V+1, W+1`)
2. **P3** ×4: stampa `D D D D`, stato `(0,0,3)` e **deadlock**
   (P1 bloccato perché `U=0`; P2 e P3 bloccati perché `V=0`).

Esercizio 204)

$$\text{Need} = \text{Max} - \text{Allocation} = \begin{pmatrix} 1 & 1 & 2 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}$$

Applichiamo il test di sicurezza (Banker's safety test) con
$\text{Available} = (2,\ 2,\ x,\ 3).$

- Per $x = 0$: nessun processo ha Need $\leq$ Available ⇒ **non sicuro**.
- Per $x = 1$: esiste una sequenza sicura, ad esempio
  $\langle T_1, T_2, T_3, T_4, T_0 \rangle$.

Quindi il **minimo** è

$$\boxed{x = 1}$$

Con $x = 1$ l'Available è $(2, 2, 1, 3)$, ma $\text{Request}_0 = (0, 0, 2, 0)$ richiede 2 unità di R_3 e quindi non è immediatamente soddisfabile.

Cerchiamo il minimo incremento Δ su R_3 tale che,
con $\text{Available}' = (2, 2, 1 + \Delta, 3)$, la richiesta sia concedibile e lo stato resti sicuro.

- Con $\Delta = 0$: impossibile (risorsa R_3 insufficiente).
- Con $\Delta = 1$:
  dopo la concessione a T_0 (Allocation_0 diventa $(1, 0, 3, 1)$, Need_0 $(1, 1, 0, 1)$, Available $(2, 2, 0, 3)$) il safety test è **positivo**, ad esempio con sequenza
  $\langle T_0, T_1, T_2, T_3, T_4 \rangle$.

Quindi il minimo incremento è

$$\boxed{\Delta = 1}$$

Esercizio 205)

```
package main
import (
	"fmt"
	"sync"
)

// worker riceve numeri da un canale, calcola il quadrato e invia il risultato in un altro canale
func worker(id int, nums <-chan int, results chan<- int, wg *sync.WaitGroup) {
	defer wg.Done() // segnala che questa goroutine ha finito al termine della funzione
	for n := range nums { // legge dal canale fino a quando è chiuso
		results <- n * n // invia il quadrato del numero sul canale dei risultati
	}
}

func main() {
	nums := make(chan int, 5)		// canale buffered per inviare i numeri ai worker
	results := make(chan int, 5) // canale buffered per raccogliere i risultati
	var wg sync.WaitGroup			// WaitGroup per sincronizzare la fine delle goroutine

	// avvia 3 worker goroutine
	for i := 0; i < 3; i++ {
		wg.Add(1) // incrementa il contatore del WaitGroup
		go worker(i, nums, results, &wg) // avvia il worker
	}
	// invia 5 numeri nel canale
	for i := 1; i <= 5; i++ {
		nums <- i
	}
	close(nums) // chiude il canale di input: i worker ora sanno che non arriveranno altri dati

	wg.Wait()		// attende che tutte le goroutine abbiano terminato
	close(results) // chiude il canale dei risultati (nessuno scriverà più)

	// stampa i risultati ricevuti
	for r := range results {
		fmt.Println(r)
	}
}
```

worker: riceve numeri da nums, calcola il quadrato e lo invia su results.
main: crea i canali, avvia 3 worker, invia 5 numeri, attende che finiscano e stampa i risultati.
wg è un puntatore a un sync.WaitGroup, una struttura che serve a sincronizzare più goroutine.
Quando chiami wg.Add(1) nel main, stai dicendo: "Attendo che una nuova goroutine termini".
Alla fine della goroutine, devi chiamare wg.Done() per dire: "Questa goroutine ha finito".
defer posticipa l'esecuzione di wg.Done() fino alla fine della funzione.
Anche se la funzione esce anticipatamente (per esempio con un return o un errore), defer garantisce che wg.Done() venga comunque eseguito.
È una buona pratica usare defer in questo caso, per evitare dimenticanze o errori che potrebbero causare un deadlock (cioè il programma resta bloccato perché il WaitGroup aspetta all'infinito).
Senza defer, potresti scrivere:
go
CopiaModifica

```
func worker(...) {
   // ... lavoro
   wg.Done() // ma se ti dimentichi di chiamarlo, il programma si blocca!
}
```

Con defer invece:
go
CopiaModifica

```
func worker(...) {
   defer wg.Done() // più sicuro e automatico
   // ... lavoro
}
```

Supponiamo:
Invii i numeri: 1, 2, 3, 4, 5 nel canale nums.
Hai 3 worker in attesa (ognuno fa range nums).
Appena un numero è nel canale, il primo worker libero lo prende e lo processa.
Non si sa quale worker prenderà quale numero: dipende da chi è libero in quel momento.

Esercizio 206) In un sistema a core singolo, ogni volta che arriva un nuovo processo, il kernel confronta il tempo rimanente del processo in esecuzione con la durata del nuovo arrivato. Se il nuovo è più breve, avviene la prelazione.
Caso a singolo core:

Processo	Arrivo (t)	Durata (Burst)
P1	0	8 ms
P2	1	4 ms
P3	2	2 ms
P4	3	1 ms

1. $t = 0$: Arriva **P1** (8ms). Inizia l'esecuzione.
2. $t = 1$: Arriva **P2** (4ms). P1 ha ancora 7ms. Poiché $4 < 7$, **P2 scalza P1**.
3. $t = 2$: Arriva **P3** (2ms). P2 ha ancora 3ms. Poiché $2 < 3$, **P3 scalza P2**.
4. $t = 3$: Arriva **P4** (1ms). P3 ha ancora 1ms. C'è un pareggio ($1 = 1$), solitamente si finisce il processo corrente. P3 continua.
5. $t = 4$: **P3 termina**. Processi pronti: P4 (1ms), P2 (3ms), P1 (7ms). Inizia **P4**.
6. $t = 5$: **P4 termina**. Inizia **P2** (3ms).
7. $t = 8$: **P2 termina**. Inizia **P1** (7ms).
8. $t = 15$: **P1 termina**.

```
Core 1: | P1 | P2 | P3 | P4 | P2 | P1 |
Time:   0    1    2    4    5    8    15
```

Caso a due core, supponiamo non esistano altri processi nel sistema oltre a quelli elencati in tabella:

1. $t = 0$: Arriva **P1**. **Core 1** lo prende in carico.
2. $t = 1$: Arriva **P2**. Il **Core 2** è libero! P2 inizia subito sul Core 2. **Nessuna prelazione su P1.**
3. $t = 2$: Arriva **P3** (2ms). Entrambi i core sono occupati.
   - Core 1: P1 (rimangono 6ms)
   - Core 2: P2 (rimangono 3ms)
   - P3 è più breve di entrambi. **P3 scalza P2** sul Core 2 (perché P2 è il più breve tra i due in esecuzione).
4. $t = 3$: Arriva **P4** (1ms).
   - Core 1: P1 (5ms)
   - Core 2: P3 (1ms)
   - P4 è più breve di P1. **P4 scalza P1** sul Core 1.
5. $t = 4$: **P3 e P4 terminano** quasi insieme. I Core 1 e 2 tornano a P2 e P1.

```
Core 1: [ P1 (1ms) ][ P4 (1ms) ][   P1 (restanti 7ms)   ]
Core 2: -[ P2 (1ms) ][ P3 (2ms) ][ P2 (restanti 3ms) ]
```

Processo	Fine (1 Core)	TT (1 Core)	Fine (2 Core)	TT (2 Core)
P1	15	15 ms	9	9ms
P2	8	7 ms	7	6 ms
P3	4	2 ms	4	2 ms
P4	5	2 ms	4	1 ms
Media		**6.5 ms**		4.5 ms

Esercizio 207)

Si verificano 3 errori nelle posizioni **1, 2 e 3**. I bit in quelle posizioni vengono invertiti ($0 \to 1$).

- Parola originale: `0 0 0 0 0 0 0`
- **Parola ricevuta:** `1 1 1 0 0 0 0`

Il ricevitore ricalcola i controlli di parità sulla sequenza ricevuta `1 1 1 0 0 0 0`

P_1(pos. 3, 5, 7): bit {1, 0, 0 }: 1 Risultato: OK, uguale alla parità della sequenza ricevuta;
P_2(pos. 3, 6, 7): bit {1, 0, 0}: 1 Risultato: OK, uguale alla parità della sequenza ricevuta;
P_4(pos. 5, 6, 7): bit (0, 0, 0}: 0 Risultato: OK, uguale alla parità della sequenza ricevuta.

Usando tale codifica per il sistema non ci sono errori. Quando nel codice di hamming con distanza minima 3 (dunque in grado di correggere un errore) allora posso scoprire fino a 2 errori (distanza minima - 1), con 3 errori ottengo altra codifica permessa.

Esercizio 208)

1. **Commutativa:** $A \oplus B = B \oplus A$ (l'ordine non conta).
2. **Associativa:** $(A \oplus B) \oplus C = A \oplus (B \oplus C)$ (le parentesi non contano).
3. **Elemento Inverso:** $A \oplus A = 0$ (qualsiasi numero combinato con se stesso si annulla).
4. **Identità:** $A \oplus 0 = A$ (lo zero è neutro).

Se definiamo la parità P come la somma XOR di tre dischi:

$$P = D1 \oplus D2 \oplus D3$$

Se il disco $D2$ si rompe, noi prendiamo tutti i dischi rimasti ($D1, D3, P$) e facciamo lo XOR tra loro:

$$(D1 \oplus D3) \oplus P$$

Sostituiamo P con la sua formula originale:

$$(D1 \oplus D3) \oplus (D1 \oplus D2 \oplus D3)$$

Per la proprietà commutativa e associativa, possiamo spostare i termini:

$$(D1 \oplus D1) \oplus (D3 \oplus D3) \oplus D2$$

E qui avviene la magia dell'inversa ($A \oplus A = 0$):

$$0 \oplus 0 \oplus D2 = D2$$

---

1. $1 \oplus 1 \oplus 0 \oplus 1 = 1$. Il bit di parità è 1.
2. Ricevuto $1001 \to 1 \oplus 0 \oplus 0 \oplus 1 = 0$. La parità è cambiata ($0 \neq 1$), errore rilevato.
3. Ricevuto $0001 \to 0 \oplus 0 \oplus 0 \oplus 1 = 1$. La parità è ancora 1! L'errore doppio non viene rilevato. Hamming invece lo rileverebbe grazie ai bit di controllo incrociati.